

RTL Design Sherpa

Bridge Hardware Architecture Specification 2.0

October 14, 2026

Table of Contents

1 Bridge Has Index....19

2 Document Information....20

2.1 Bridge Hardware Architecture Specification....20

2.2 Revision History....21

2.3 Document Purpose....25

2.4 Intended Audience....26

2.5 Related Documents....27

3 Purpose and Scope....28

3.1 What is Bridge?....28

3.2 The Problem Bridge Solves....29

3.3 Scope....30

3.4 Out of Scope....31

4 Document Conventions....32

4.1 Terminology....32

4.2 Signal Naming....33

4.3 Diagrams....34

4.3.1 Figure 1.1: Block Diagram Legend....34

4.4 Code Examples....35

4.5 Parameter Notation....36

5 Definitions and Acronyms....37

5.1 Acronyms....37

5.2 Definitions....39

6 Use Cases 40
6.1 Primary Applications 40
6.1.1 SoC Interconnects 40
6.1.2 Memory Systems 40
6.1.3 Accelerator Systems 40
6.2 Example: RAPIDS Accelerator 41
6.2.1 Figure 2.1: RAPIDS System Topology 41
6.2.2 Configuration 41
6.2.3 Benefits 41
6.3 Rapid Prototyping 43
7 Key Features 44
7.1 Configuration-Driven Generation 44
7.1.1 CSV/TOML Configuration 44
7.1.2 Example Configuration 44
7.2 Multi-Protocol Support 45
7.2.1 Protocol Features 46
7.3 Channel-Specific Masters 47
7.3.1 Write-Only Masters (wr) 47
7.3.2 Read-Only Masters (rd) 47
7.3.3 Full Masters (rw) 47
7.3.4 Resource Savings 47
7.4 Automatic Converters 48
7.4.1 Width Conversion 48
7.4.2 Protocol Conversion 48

7.5 Flexible Topology 49
7.5.1 Scalability 49
7.5.2 Mixed Configurations 49
8 System Context 50
8.1 Bridge in SoC Architecture 50
8.1.1 Figure 2.2: Bridge System Context 50
8.2 Interface Boundaries 51
8.2.1 Master-Side (Upstream) 51
8.2.2 Slave-Side (Downstream) 51
8.3 Address Space 52
8.3.1 Address Map Organization 52
8.3.2 Address Decode 52
8.4 Clock and Reset 53
8.4.1 Clock Domain 53
8.4.2 Reset 53
8.5 Dependencies 54
8.5.1 Required Infrastructure 54
8.5.2 Optional Features 54
8.6 References 55
9 Block Diagram 56
9.1 Bridge Architecture Overview 56
9.1.1 Figure 3.1: Bridge Block Diagram 56
9.2 Functional Blocks 57
9.2.1 Master Adapter 57

9.2.2 Crossbar Core....57
9.2.3 Slave Adapter....57
9.3 Data Flow....58
9.3.1 Figure 3.2: Write Path....58
9.3.2 Figure 3.3: Read Path....59
9.4 Scalability....60
9.4.1 Topology Parameters....60
9.4.2 Resource Scaling....60
9.5 References....61
10 Data Flow....62
10.1 Transaction Flow Overview....62
10.1.1 Write Transaction Flow....62
10.1.2 Read Transaction Flow....62
10.2 Channel Independence....63
10.2.1 AXI4 Channel Separation....63
10.2.2 Parallel Operation....63
10.3 Bridge ID and ID Widening....64
10.3.1 Bridge ID (BID) Concept....64
10.3.2 IDs are widened inside the fabric....64
10.4 Response Routing....65
10.4.1 B Channel (Write Response)....65
10.4.2 R Channel (Read Data)....65
10.4.3 The requirement the FIFO form creates....66
10.5 Ordering: what the fabric actually guarantees....67

10.6 References....68
11 Protocol Support....69
11.1 Supported Protocols....69
11.1.1 AXI4 Full....69
11.1.2 AXI4-Lite....69
11.1.3 AXI3....69
11.1.4 APB (Advanced Peripheral Bus)....69
11.2 Protocol Conversion Matrix....71
11.3 Automatic Conversion Shims at Slave Boundary....73
11.4 Automatic Conversion at the Master Boundary....74
11.5 AXI4 to APB Conversion....76
11.5.1 Conversion Process....76
11.5.2 Figure 3.4: AXI4 to APB Conversion Sequence....76
11.5.3 Burst Handling....76
11.5.4 Timing Considerations....76
11.6 Width Conversion....77
11.6.1 Upsize (Narrow to Wide)....77
11.6.2 Figure 3.5: Width Upsize Conversion....77
11.6.3 Downsize (Wide to Narrow)....78
11.6.4 Figure 3.6: Width Downsize Conversion....78
11.7 Channel-Specific Protocol....79
11.7.1 Write-Only Masters....79
11.7.2 Read-Only Masters....79
11.8 ACE and Coherency — Explicitly Out of Scope....80

11.9 References....81
12 AXI4 Interface Overview....82
12.1 Interface Organization....82
12.1.1 Master-Side Interfaces....82
12.1.2 Slave-Side Interfaces....82
12.2 Signal Naming Convention....84
12.2.1 Custom Prefixes....84
12.2.2 ID Width Extension....84
12.3 Channel-Specific Interfaces....85
12.3.1 Write-Only Master (wr)....85
12.3.2 Read-Only Master (rd)....85
12.4 Monitor Bus Output (when variants includes mon)....86
12.5 Handshake Protocol....87
12.5.1 Valid/Ready Handshake....87
12.5.2 Backpressure....87
12.6 Burst Support....88
12.6.1 Supported Burst Types....88
12.6.2 Burst Length....88
13 APB Interface....89
13.1 Overview....89
13.2 Ports....90
13.2.1 Signal Definition....90
13.2.2 Signal Naming....90
13.3 Functional Description....91

13.3.1 AXI4 to APB Conversion....91
13.3.2 Burst Handling....91
13.3.3 Error Mapping....91
13.4 Timing....92
13.5 Waveforms....93
13.5.1 Write Transaction....93
13.5.2 Read Transaction....94
13.6 APB Requester Ports....95
13.7 Design Notes....96
14 Clock and Reset....97
14.1 Overview....97
14.2 Ports....98
14.2.1 Clock....98
14.2.2 Reset....98
14.3 Functional Description....99
14.3.1 One Fabric Clock....99
14.3.2 Clock Domain Crossing: CDC Slave Ports....99
14.3.3 Reset Behavior....101
14.3.4 Reset Effects....101
14.3.5 Outstanding Transactions....101
14.3.6 Reset Release....101
14.4 Timing....102
14.4.1 Setup and Hold....102
14.4.2 Clock-to-Output....102

14.5 Usage Example..... 103
14.5.1 Recommended Constraints..... 103
14.6 Design Notes..... 104
14.6.1 External CDC Required..... 104
14.6.2 CDC Recommendations..... 104
15 AXI5 and APB5 Interfaces (AMBA5 Support)..... 105
15.1 Overview..... 105
15.2 Parameters..... 106
15.2.1 Declaring AMBA5 Ports..... 106
15.3 Ports..... 107
15.3.1 AXI5 Port Surface..... 107
15.3.2 APB5 Slave Surface..... 108
15.3.3 AXI5-Lite Master Surface..... 108
15.3.4 APB5 Master Surface..... 109
15.3.5 AXI5-Lite Slave Surface..... 110
15.4 Functional Description..... 111
15.4.1 Interop Matrix..... 113
16 Unmapped-Address Handling..... 114
16.1 Overview..... 114
16.1.1 Why the bridge answers an address no slave owns..... 114
16.2 Ports..... 115
16.2.1 Status, interrupt and clearing..... 115
16.3 Functional Description..... 116
16.3.1 Behaviour..... 116

16.3.2 Register access (cfg regblock builds)....116
16.3.3 When it can never fire....117
16.3.4 What this does not do....117
16.4 Navigation....118
17 Wishbone B4 Interface....119
17.1 Overview....119
17.2 Ports....120
17.2.1 Signal Definition....120
17.2.2 Signal Naming....121
17.3 Functional Description....122
17.3.1 Slave port: AXI4 to Wishbone....122
17.3.2 Master port: Wishbone to AXI4....122
17.3.3 Rules the validator enforces....122
17.4 Design Notes....123
18 Throughput Characteristics....124
18.1 Overview....124
18.2 Functional Description....125
18.2.1 Peak Throughput....125
18.2.2 Formula....125
18.2.3 Measured (2026-09-11)....125
18.2.4 Factors Affecting Throughput....126
18.2.5 Multi-Master Scaling....127
18.2.6 Burst Efficiency....127
18.2.7 Width Conversion Impact....127

18.2.8 Protocol Conversion Impact....128
18.3 Design Notes....129
18.4 References....130
19 Latency Analysis....131
19.1 Overview....131
19.2 Timing....132
19.2.1 Address Path Latency....132
19.2.2 Data Path Latency....132
19.2.3 Response Path Latency....132
19.2.4 Loaded Write Latency (measured)....133
19.2.5 End-to-End Latency....134
19.2.6 Width Conversion Latency....135
19.2.7 Protocol Conversion Latency....135
19.3 Design Notes....136
19.3.1 Design Recommendations....136
19.4 References....137
20 Resources....138
20.1 Overview....138
20.2 Measured Utilization and Timing....139
20.2.1 Scaling....141
20.3 Design Notes....142
20.3.1 Resource Reduction....142
20.3.2 Performance/Resource Trade-off....142
20.4 References....143

21 System Requirements....144
21.1 Overview....144
21.1.1 Clock Infrastructure....144
21.1.2 Reset Infrastructure....144
21.1.3 Power....144
21.2 Parameters....145
21.2.1 Address Map....145
21.2.2 Connectivity....145
21.3 Ports....146
21.3.1 Master Requirements....146
21.3.2 Slave Requirements....146
21.4 Timing....147
21.4.1 Setup/Hold....147
21.4.2 Recommended Margins....147
21.5 Testing....148
21.5.1 Pre-Integration Checks....148
21.5.2 Integration Checks....148
21.5.3 Post-Integration Checks....148
21.6 Assumptions and Known Limitations....149
21.6.1 Response-Ordering Assumption....149
21.6.2 Buffer and CAM Depths....149
21.7 References....150
22 Parameter Configuration....151
22.1 Overview....151

22.2 Parameters....152
22.2.1 Core Parameters....152
22.2.2 Registered Crossbar (xbar_pipeline)....152
22.2.3 Derived Parameters....155
22.2.4 Per-Port Configuration....156
22.2.5 Top-Level Monitor Configuration....158
22.2.6 Parameter Validation....159
22.3 Usage Example....160
22.3.1 TOML Configuration....160
22.3.2 CSV Connectivity....160
22.3.3 Simple 2x2....160
22.3.4 Mixed Protocol....161
23 Verification Strategy....162
23.1 Overview....162
23.2 Testing....163
23.2.1 Verification Levels....163
23.2.2 Test Infrastructure....164
23.2.3 Test Execution Model....165
23.2.4 Coverage Goals....165
23.2.5 Debug Features....166
23.2.6 Regression Testing....167
23.2.7 Known Limitations....167
24 Synthesis and Implementation....169
24.1 Overview....169

24.2 The Flow....170
24.2.1 Where it lives....170
24.2.2 How the bridge is constrained....171
24.2.3 Reading the summary line....171
24.3 Measured Results....173
24.3.1 What the first run found....173
24.3.2 The reference set....173
24.3.3 Reading the rows....175
24.4 Design Notes....177
24.4.1 What moves the numbers....177
24.4.2 Before a board build....177
25 AXI3 Integration....178
25.1 Boundary-Conversion Architecture....178
25.2 Converter-Sourced Constraints....179
25.3 Validation Evidence....180
25.3.1 Configuration constraints....180
25.4 References....181

List of Figures

Figure 1.1: Block Diagram Legend....34
Figure 2.1: RAPIDS System Topology....41
Figure 2.2: Bridge System Context....50
Figure 3.1: Bridge Block Diagram....56
Figure 3.2: Write Path....58
Figure 3.3: Read Path....59
Figure 3.4: AXI4 to APB Conversion Sequence....76
Figure 3.5: Width Upsize Conversion....77
Figure 3.6: Width Downsize Conversion....78
Figure 4.1: APB Write Transfer (Zero Wait States)....93
Figure 4.2: APB Read Transfer (One Wait State)....94
Figure 4.3: Clock Distribution....99
Figure 4.4: Reset Timing....101
Figure 4.5: Multi-Clock CDC....104

List of Tables

Table 1: Document information....20
Table 2: Revision history....21
Table 3: Common Terminology....32
Table 4: Signal Naming Prefixes....33
Table 5: Block Diagram Symbols....34
Table 6: Parameter Format Example....36
Table 7: Acronyms and Abbreviations....37
Table 8: Supported Protocols....45
Table 9: Channel-Specific Resource Savings....47
Table 10: Address Map Organization....52
Table 11: Topology Parameter Scaling....60
Table 12: AXI4 Channel Summary....63
Table 13: Response Routing....65
Table 14: Protocol Conversion Matrix....71
Table 15: Slave-port protocol values....73
Table 16: Master-port protocol values....74
Table 17: Master-Side Interface Channels....82
Table 18: Slave-Side Interface Channels....83
Table 19: Monitor Bus Output....86
Table 20: Supported Burst Types....88
Table 21: APB Signal Definitions....90
Table 22: AXI4 to APB Burst Conversion....91
Table 23: APB to AXI4 Error Mapping....91
Table 24: Clock Requirements....98
Table 25: Reset Signal....98
Table 26: CDC slave port clock pins (cdc = true)....100
Table 27: Reset Effects....101
Table 28: Timing Constraints....102
Table 29: Clock-to-Output Delays....102
Table 30: AXI5 Port Surface....107
Table 31: APB5 Slave Surface....108
Table 32: AXI5-Lite sideband groups at an axil5 master boundary....109
Table 33: AXI5-Lite sideband groups at an axil5 slave boundary....110
Table 34: Interop Matrix....113
Table 35: Unmapped-address status and interrupt ports....115
Table 36: Subtractive-slave response behaviour....116

Table 37: Unmapped-address status registers....116
Table 38: Wishbone B4 Signal Definitions....120
Table 39: Wishbone termination to AXI4 response....122
Table 40: Peak Throughput by Data Width....125
Table 41: Measured throughput, direct 32-bit AXI4 paths....125
Table 42: Factors Affecting Throughput....126
Table 43: Burst Efficiency Comparison....127
Table 44: AXI4 vs APB Protocol Impact....128
Table 45: Address Path Latency....132
Table 46: Data Path Latency....132
Table 47: Response Path Latency....132
Table 48: Write latency under load, bridge_2x2_rw, 32-bit direct path....134
Table 49: Upsize Latency by Ratio....135
Table 50: Downsize Latency by Ratio....135
Table 51: AXI4 to APB Conversion Latency....135
Table 52: Measured utilization and timing, Artix-7 100T -1 at 10 ns (HAS 6.4 flow)
....139
Table 53: Measured utilization and timing, Kintex-7 325T -2 at 6.667 ns....139
Table 54: Resource Scaling....141
Table 55: Performance/Resource Trade-offs....142
Table 56: Clock Requirements....144
Table 57: Reset Requirements....144
Table 58: Power Requirements....144
Table 59: Address Map Requirements....145
Table 60: Connectivity Requirements....145
Table 61: Timing Requirements....147
Table 62: Recommended Timing Margins....147
Table 63: Bridge Core Parameters....152
Table 64: Crossbar options (bridge TASK-006, was BRIDGE-017)....152
Table 65: CDC slave port option (bridge TASK-006, was BRIDGE-017)....154
Table 66: Monitor preset....154
Table 67: Derived Parameters....156
Table 68: Master Port Configuration....156
Table 69: Slave Port Configuration....157
Table 70: Top-Level Monitor Configuration....158
Table 71: Generator Validation Checks....159
Table 72: Unit Level Test Focus....163
Table 73: Integration Test Coverage....163

Table 74: System Level Tests....163
Table 75: Test Categories....164
Table 76: Functional Coverage Goals....165
Table 77: Code Coverage Goals....166
Table 78: Debug Signals....166
Table 79: CI Test Schedule....167
Table 80: Configuration Test Matrix....167
Table 81: Known Test Gaps....167
Table 82: Simulator Support Status....167
Table 83: Synthesis flow knobs....170
Table 84: summary.csv columns....171
Table 85: The CAM ordering-count scan, before and after (2026-09-11)....173
Table 86: Reference set on the Artix-7 100T -1 (Nexys A7), constrained at 10 ns. 173
Table 87: Reference set on the Kintex-7 325T -2 (Genesys 2), constrained at 6.667 ns....174

1 Bridge Has Index

Generated: 2026-10-14

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Document Information

2.1 Bridge Hardware Architecture Specification

Document information

Property	Value
Document Title	Bridge Hardware Architecture Specification
Version	2.0
Date	September 8, 2026
Status	Released
Classification	Open Source - MIT License

2.2 Revision History

Revision history

Version	Date	Author	Description
1.0	2026-01-03	RTL Design Sherpa	Initial release - restructured from single spec
1.1	2026-06-04	RTL Design Sherpa	Content sync to RTL state at 2026-06-04. Documents (1) the 64→128-bit monbus packet migration with new 64-bit side-band timestamp wire — referenced via `docs/markdown/rtl-amba/includes/ocelus_package_spec.md`; (2) per-port + global SV-parameter monitor methodology with mandatory `use_monitor` TOML field and `variants = ["no","mon"]` generator selector; (3) generator-emitted AXI4↔️AXIL conversion at the slave boundary (was external); (4) new AXIL→wider-slave master-side alignment converters (`axil_to_axi4_wide_align_{rd,wr}`); (5) verification-strategy rewrite to protocol-BFM-only TBs with memory-backed slaves, serial-only execution, per-module cocotb function names, boundary_probe (renamed

Version	Date	Author	Description
			from address_decode) tests; (6) note of the independent W-tracker FIFO + split AW/W-ready MUX fix (commit `d24bd617`) that closed an interleaved-master deadlock.
1.2	2026-09-08	RTL Design Sherpa	Correctness and voice pass. Four external qc rounds (65/54/53/57 findings, all triaged against rtl/generated and the generator) removed the planned-but-never-built ID-table/CAM out-of-order scheme, the fixed 64-bit internal path, response arbitration/FIFOs and the AXI4-Lite-as-future-work status from the text; annotated the TOML keys the loader has never known and its three-tier unknown-key behaviour; replaced asserted latency figures with measured ones. Four RTL defects found by the rounds are fixed and mutation-checked (bridge BUG-009 (was BRIDGE-011) response-FIFO overrun, axi4_subtractive_slave simultaneous AW+AR fault loss, axi5_atomic_filter duplicate B and B-before-

Version	Date	Author	Description
			WLAST). Prose then voice-passed with all correctness annotations preserved.
1.3	2026-09-11	RTL Design Sherpa	bridge TASK-004 (was BRIDGE-014): AXI5-Lite and APB/APB5 as master protocols (front-end converters); bridge TASK-005 (was BRIDGE-016) slave ID widening
1.4	2026-09-11	RTL Design Sherpa	bridge TASK-008 (was BRIDGE-019): Wishbone B4 on both sides of the fabric
1.5	2026-09-11	RTL Design Sherpa	bridge TASK-006 (was BRIDGE-017): measured throughput and latency, registered crossbar, QoS arbitration, CDC slave ports, out-of-context synthesis flow and measured resources, CAM ordering-count fix
1.6	2026-09-13	RTL Design Sherpa	bridge TASK-007 (was BRIDGE-018): native AXI5 fabric – Memory Tagging and read-data chunking through the structs
1.7	2026-09-13	RTL Design Sherpa	Document sync: one fabric clock plus CDC ports throughout, Wishbone best-effort stance, out-of-order chunk proof
2.0	2026-10-12	RTL Design Sherpa	Protocol-set correction: overview/summary bullets now state the full eight-

Version	Date	Author	Description
			protocol support (axi3/axi4/axi5/axil/axil5/apb/apb5/wb4 — per-port, slave or master side) verified against `bin/bridge_pkg/config_validator.py`; explicit “ACE coherency NOT supported” section added to ch03. Version strings aligned across index/front-matter/renders.

2.3 Document Purpose

This Hardware Architecture Specification (HAS) is the high-level view of the Bridge component — the map you read before you integrate, not the schematic. It covers:

- System-level architecture and data flow
- Protocol support (AXI3, AXI4, AXI5, AXI4-Lite, AXI5-Lite, APB, APB5, Wishbone B4 — per-port, slave or master side; ACE coherency NOT supported)
- Interface definitions and requirements
- Performance characteristics
- Integration guidelines

For micro-architecture, FSM designs, and signal-level detail, see the companion **Bridge Micro-Architecture Specification (MAS)**.

2.4 Intended Audience

- System architects defining interconnect topology
- Hardware engineers integrating Bridge into SoC designs
- Verification engineers planning test strategies
- Technical managers evaluating Bridge capabilities

2.5 Related Documents

- **Bridge MAS** - Micro-Architecture Specification (detailed block-level)
- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI development guide

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 Purpose and Scope

3.1 What is Bridge?

Bridge is a Python-based multi-protocol crossbar generator. You describe the interconnect in human-readable configuration files; it hands you back parameterized SystemVerilog RTL. The generated AXI4 crossbars come with automatic support for:

- **Protocol conversion** - AXI3, AXI4, AXI5, AXI4-Lite, AXI5-Lite, APB, APB5, and Wishbone B4 interfaces, on slave or master ports (selected per port in the TOML; ACE coherency is NOT supported)
- **Width conversion** - Automatic upsize/downsize for data width mismatches
- **Channel-specific masters** - Write-only, read-only, or full AXI4 masters

3.2 The Problem Bridge Solves

A hand-written AXI4 crossbar is error-prone. Building one means:

1. Writing 5 separate channel multiplexers (AW, W, B, AR, R)
2. Implementing per-slave arbitration for each channel
3. Routing responses back to the issuing master, including slaves that complete different IDs out of order
4. Handling burst locking and interleaving constraints
5. Inserting width converters for data width mismatches
6. Inserting protocol converters for mixed systems — APB/APB5, AXI4-Lite/AXI5-Lite, AXI3, or Wishbone B4 peripherals behind an AXI4 fabric (and vice versa)
7. Wiring hundreds of signals with consistent naming

Every item on that list is a place a bug can sit quietly until integration. I've chased most of them at least once.

Bridge automates all of it:

- Define ports and connectivity in configuration files
- Run the generator script
- Get production-ready SystemVerilog RTL

3.3 Scope

This specification covers:

- High-level architecture and block organization
- Supported protocols and conversion capabilities
- Interface requirements and signal conventions
- Performance characteristics and resource estimates
- Integration guidelines and verification strategy

3.4 Out of Scope

These live in the companion MAS:

- Detailed FSM state diagrams and transitions
- Signal-level timing diagrams
- Internal pipeline structures
- RTL implementation details

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

4 Document Conventions

4.1 Terminology

Common Terminology

Term	Definition
Master	AXI4 initiator that issues transactions
Slave	AXI4 target that responds to transactions
Crossbar	NxM interconnect connecting masters to slaves
Channel	AXI4 communication path (AW, W, B, AR, R)
Converter	Logic that transforms width or protocol

4.2 Signal Naming

Bridge uses consistent signal prefixes — learn them once and every generated interface reads the same way:

Signal Naming Prefixes

Prefix	Meaning
`s_*`	Slave-side interface (master port on bridge)
`m_*`	Master-side interface (slave port on bridge)
`axi4_*`	AXI4 protocol signals
`apb_*`	APB protocol signals
`cfg_*`	Configuration signals
`dbg_*`	Debug signals

4.3 Diagrams

4.3.1 Figure 1.1: Block Diagram Legend

Block diagrams in this specification use these symbols:

Block Diagram Symbols

Symbol	Meaning
White rectangle	External port or interface (masters at the top, slaves at the bottom)
Shaded rectangle	Functional block; the second line names the owning RTL
Solid arrow	A flow, labeled with what it carries: requests down, responses back up their own lane
Heavy arrow	The main flow out of a decision point, or a path on a clock other than `aclk`
Dashed arrow	Sideband: tracking writes, configuration, clocks
Dashed box	Optional or build-dependent block
Trapezium, wide edge up	A many-to-one pick (the per-slave arbiter): contenders enter the wide edge

4.4 Code Examples

SystemVerilog examples are formatted like this:

```
// Module instantiation example
bridge_4x4 u_bridge (
    .aclk       (sys_clk),
    .aresetn    (sys_rst_n),
    // ... port connections
);
```

4.5 Parameter Notation

Parameter tables throughout the spec use this format:

Parameter Format Example

Parameter	Type	Range	Default	Description
NUM_MASTERS	int	1-32	2	Number of master ports
NUM_SLAVES	int	1-256	2	Number of slave ports

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

5 Definitions and Acronyms

5.1 Acronyms

Acronyms and Abbreviations

Acronym	Definition
AMBA	Advanced Microcontroller Bus Architecture
APB	Advanced Peripheral Bus
AR	AXI4 Read Address channel
AW	AXI4 Write Address channel
AXI	Advanced eXtensible Interface
AXI4	AXI Protocol Version 4
B	AXI4 Write Response channel
BID	Bridge ID (internal master identifier)
CAM	Content-Addressable Memory
CDC	Clock Domain Crossing
CSV	Comma-Separated Values
DECERR	Decode Error (AXI response)
DMA	Direct Memory Access
FIFO	First-In First-Out buffer
FSM	Finite State Machine
HAS	Hardware Architecture Specification
ID	Transaction Identifier
MAS	Micro-Architecture Specification
MUX	Multiplexer
NxM	N masters by M slaves (topology notation)
OOO	Out-of-Order
OOR	Out-of-Range
QoS	Quality of Service

Acronym	Definition
R	AXI4 Read Data channel
RTL	Register-Transfer Level
SLVERR	Slave Error (AXI response)
SoC	System-on-Chip
TOML	Tom’s Obvious Minimal Language
W	AXI4 Write Data channel

5.2 Definitions

Address Decode: Determining which slave receives a transaction, from its address.

Arbitration: Selecting which master gets a slave when several masters contend for it.

Bridge ID (BID): The issuing master's index, width clog2(NUM_MASTERS). With more than one master it is both prepended to the AXI ID inside the fabric (slave-side IDs are `{BRIDGE_ID, id}`, wider than master-side ones) and carried as a sideband into the slave adapter's response tracker (a `bridge_cam` or an in-order FIFO), which routes B and R back to the master.

Channel-Specific Master: A master that uses only a subset of AXI4 channels (write-only or read-only).

Crossbar: An NxM interconnect allowing any master to communicate with any connected slave.

Grant Locking: Mechanism that holds arbitration grant until transaction completes.

Out-of-Order Response: (defined for completeness; NOT supported by this bridge) Responses returned in different order than requests were issued.

Protocol Conversion: Transformation between different bus protocols (e.g., AXI4 to APB).

Response Routing: Directing B/R channel responses back to the originating master.

Round-Robin Arbitration: Fair arbitration where priority rotates among requesters.

Width Conversion: Transformation between different data bus widths (upsize or downsize).

6 Use Cases

6.1 Primary Applications

6.1.1 SoC Interconnects

In SoC designs, Bridge is the main interconnect fabric:

- **Multi-core processor memory subsystems** - Connect multiple CPU cores to shared memory
- **Accelerator integration** - Route GPU, DSP, or custom accelerator traffic
- **DMA engine routing** - Connect DMA controllers to memory and peripherals
- **Peripheral bus bridges** - Convert AXI4 to APB for low-speed peripherals

6.1.2 Memory Systems

Typical memory-system uses:

- **Multi-port DDR controllers** - Multiple masters accessing shared DDR
- **SRAM buffer sharing** - Shared packet buffers for networking
- **Cache coherency interconnects** - Connect cache controllers
- **Memory-mapped I/O** - Unified address space for peripherals

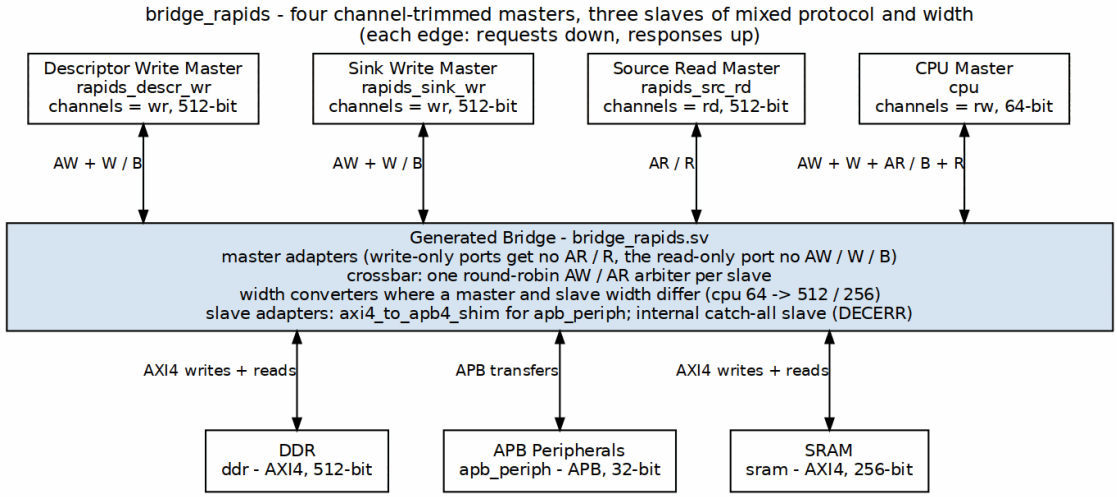
6.1.3 Accelerator Systems

Bridge supports custom accelerator architectures like RAPIDS:

- **Descriptor write masters** - Write control descriptors (write-only)
- **Sink write masters** - Write incoming data packets (write-only)
- **Source read masters** - Read outgoing data packets (read-only)
- **CPU configuration** - Full access for register configuration

6.2 Example: RAPIDS Accelerator

6.2.1 Figure 2.1: RAPIDS System Topology



RAPIDS System Topology

Bridge topology for RAPIDS accelerator with channel-specific masters.

6.2.2 Configuration

```
[bridge]
  name = "bridge_rapids"
  masters = [
    {name = "rapids_descr_wr", channels = "wr", data_width = 512},
    {name = "rapids_sink_wr", channels = "wr", data_width = 512},
    {name = "rapids_src_rd", channels = "rd", data_width = 512},
    {name = "cpu", channels = "rw", data_width = 64}
  ]
  slaves = [
    {name = "ddr", protocol = "axi4", data_width = 512},
    {name = "apb_periph", protocol = "apb", data_width = 32},
    {name = "sram", protocol = "axi4", data_width = 256}
  ]
```

6.2.3 Benefits

- **40% fewer channels** for write-only masters (3 of 5: AW, W, B)
- **60% fewer channels** for read-only masters (2 of 5: AR, R)

Those are CHANNEL counts, which is the honest way to state it – the saving in actual wires depends on the widths. At DW=64/AW=32/IW=4/UW=1 the five channels are roughly 295 bits, and dropping AR+R saves about 48% of them while dropping AW+W+B saves about 52%; at DW=512 it is nearer 47%/53%. Earlier revisions of this page quoted “39%” and “61%”, which match neither

basis. - **Automatic width conversion** for 64b CPU to 512b DDR - **Protocol conversion** for APB peripheral access

6.3 Rapid Prototyping

Bridge accelerates design exploration:

- **Quick topology exploration** - Try different master/slave configurations
- **Performance analysis** - Evaluate arbitration and bandwidth
- **Design space exploration** - Compare resource usage
- **Educational projects** - Learn AXI4 protocol and interconnects

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

7 Key Features

7.1 Configuration-Driven Generation

7.1.1 CSV/TOML Configuration

Everything about a Bridge instance is set in human-readable text files:

- **ports configuration** - Define each master and slave port
- **connectivity matrix** - Specify which masters connect to which slaves
- **Version-control friendly** - Text-based, easy to diff and review

7.1.2 Example Configuration

Port Definition (TOML):

```
masters = [
  {name = "cpu", prefix = "cpu_m_axi", data_width = 64, channels =
"rw"},
  {name = "dma", prefix = "dma_m_axi", data_width = 256, channels =
"rw"}
]
slaves = [
  {name = "ddr", prefix = "ddr_s_axi", data_width = 512, protocol =
"axi4"},
  {name = "uart", prefix = "uart_apb", data_width = 32, protocol =
"apb"}
]
```

Connectivity Matrix (CSV):

```
master\slave,ddr,uart
cpu,1,1
dma,1,0
```

7.2 Multi-Protocol Support

Supported Protocols

Master Protocol	Slave Protocol	Conversion Type
AXI4	AXI4	Direct or width convert
AXI4	AXI4-Lite	Protocol downgrade
AXI4	APB	Full protocol conversion
AXI5	AXI4 / AXI4-Lite / APB	Base-subset interop; sideband terminates at the boundary (warning)
AXI5	AXI5 (width-matched)	Native sideband pass-through; atomics (read-return classes need an rw port); poison
AXI4 / AXI5	APB5	APB4 conversion core + APB5 sideband surface
AXI4-Lite / AXI5-Lite	any	Lite requester: single-beat AXI4 inside; the AXI5-Lite ID and `user` group ride the fabric, the rest terminate at the boundary
APB / APB5	any	APB requester: `apb4_to_axi4` / `apb5_to_axi4` front end, one AXI4 transaction per transfer, SLVERR and DECERR fold to PSLVERR
any	Wishbone B4	`axi4_to_wb4`: one Wishbone transfer per AXI4 beat, ERR/RTY fold to SLVERR
Wishbone B4	any	`wb4_to_axi4` front end: single-beat AXI4 inside, SLVERR/DECERR terminate ERR

AMBA5 ports are declared per port (`protocol = "axi5"` / `"axil5"` / `"apb5"` with an optional `axi5_features` list); the fabric stays AXI4 internally. Every protocol value is legal on a master port as well as a slave port (bridge TASK-004, was BRIDGE-014). See AXI5 and APB5 Interfaces.

7.2.1 Protocol Features

- **AXI4 Full** - Complete 5-channel implementation with bursts
- **AXI4-Lite** - Simplified single-beat transactions
- **APB** - Low-power peripheral access
- **Wishbone B4** - The open FPGA bus, pipelined mode, on either side (bridge TASK-008, was BRIDGE-019)

7.3 Channel-Specific Masters

7.3.1 Write-Only Masters (wr)

Generate only write channels: - AW (Write Address) - W (Write Data) - B (Write Response)

Use case: DMA write engines, descriptor writers

7.3.2 Read-Only Masters (rd)

Generate only read channels: - AR (Read Address) - R (Read Data)

Use case: DMA read engines, source data fetchers

7.3.3 Full Masters (rw)

Generate all five channels for complete read/write capability.

Use case: CPU interfaces, configuration masters

7.3.4 Resource Savings

Channel-Specific Resource Savings

Master Type	Channels	Signal Reduction
Write-only (wr)	3 of 5	40% fewer channels (~48% fewer wires at DW=64)
Read-only (rd)	2 of 5	60% fewer channels (~52% fewer wires at DW=64)
Full (rw)	5 of 5	Baseline

7.4 Automatic Converters

7.4.1 Width Conversion

Bridge automatically inserts width converters:

- **Upsize** - Narrow master to wide slave (e.g., 64b to 512b)
- **Downsize** - Wide master to narrow slave (e.g., 512b to 32b)
- **Per-path** - Only where needed, not global conversion

7.4.2 Protocol Conversion

Bridge inserts protocol converters for mixed systems:

- **AXI4 to APB** - Full conversion with burst handling
- **AXI4 to AXI4-Lite** - Simplified conversion

7.5 Flexible Topology

7.5.1 Scalability

- **Masters:** 1-32 master ports
- **Slaves:** 1-256 slave ports
- **Partial connectivity** - Not all masters need access to all slaves

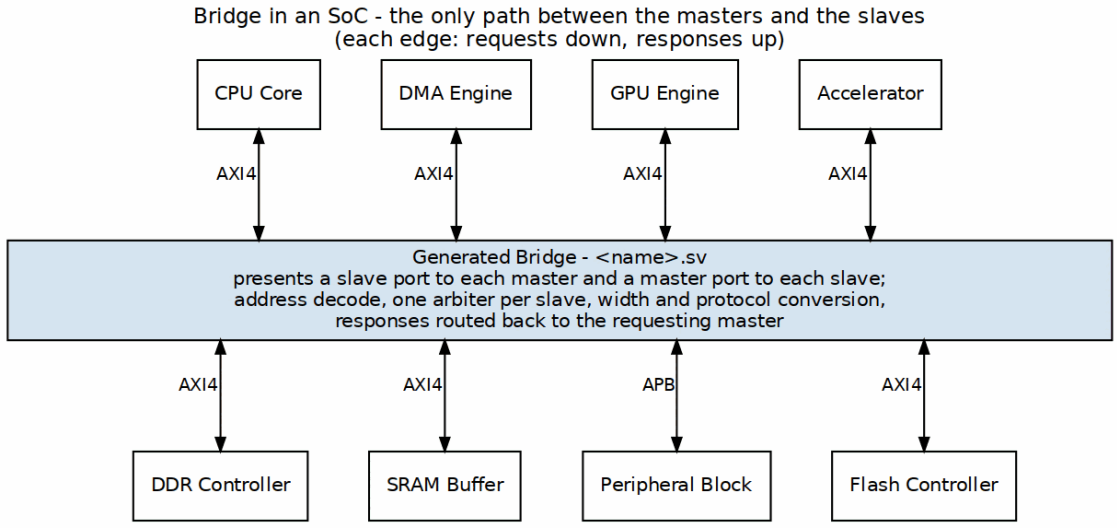
7.5.2 Mixed Configurations

- **Mixed protocols** - AXI4, AXI4-Lite, and APB slaves in same crossbar
- **Mixed data widths** - 32b, 64b, 128b, 256b, 512b
- **Mixed channels** - Write-only, read-only, and full masters

8 System Context

8.1 Bridge in SoC Architecture

8.1.1 Figure 2.2: Bridge System Context



Bridge System Context

Bridge as central interconnect connecting masters to slaves with protocol conversion.

8.2 Interface Boundaries

8.2.1 Master-Side (Upstream)

Bridge presents slave interfaces to upstream masters:

- Accepts AXI4 transactions from masters
- Provides flow control via ready signals
- Returns responses (B/R channels) to correct master

8.2.2 Slave-Side (Downstream)

Bridge presents master interfaces to downstream slaves:

- Issues AXI4/APB transactions to slaves
- Accepts responses from slaves
- Routes responses back through ID tracking

8.3 Address Space

8.3.1 Address Map Organization

Each slave occupies a configurable address region:

Address Map Organization

Slave	Base Address	Address Range	Size
Slave 0	`base_addr` from its own TOML entry	`addr_range`	Variable
Slave 1	`base_addr` from its own TOML entry	`addr_range`	Variable
Slave N	`base_addr` from its own TOML entry	`addr_range`	Variable

Windows are not derived from a single BASE and are not contiguous. Each slave carries its own `base_addr`, subject only to 4 KB alignment and non-overlap (Parameters). An earlier revision of this table showed `BASE + addr_range_0` and "sum of previous ranges", which no shipped config follows – `bridge_mix_d` places a 1 GB `ddr` at `0x0000_0000`, a 4 KB `doorbell` at `0x4000_0000` and a 64 KB `apb_periph` at `0x4000_1000`, and gaps between windows are legal (they decode-miss).

8.3.2 Address Decode

- **Subtractive decode** - the ranges are tested in order and the chain ends in an `else`, so the one-hot result is *never* all-zero
- **One-hot result** - exactly one slave selected per transaction, always
- **Out-of-range** - an address matching no slave range selects the internal subtractive slave, which completes the transaction with **DECERR** and `0xDEADBEEF` read data rather than leaving it unanswered

Until 2026-09-07 the last bullet was aspirational: the decode chain had no `else`, so an unmapped address produced an all-zero select, no slave ever saw AWVALID/ARVALID, READY never rose, and **the master waited forever**. A hang is the worst available failure here because it destroys the evidence – there is no response to inspect, no error bit to read, and the offending address is whatever the master still has latched. See 4.5 for the status/interrupt path that now reports it.

8.4 Clock and Reset

8.4.1 Clock Domain

- **One fabric clock** - All Bridge logic and every master port synchronous to `aclk`
- **CDC slave ports** - An AXI4 slave declared `cdc = true` runs on its own `<slave>_aclk` behind an async-FIFO crossing (HAS 4.5a); other slaves share `aclk`

8.4.2 Reset

- **Active-low asynchronous** - Standard `aresetn` convention
- **Full reset** - All internal state cleared on reset
- **Transaction safety** - Outstanding transactions aborted on reset

8.5 Dependencies

8.5.1 Required Infrastructure

- AXI4-compliant masters and slaves
- Proper clock and reset distribution
- Address map configuration matching slave address ranges

8.5.2 Optional Features

- Width converters (only if widths mismatch)
- Protocol converters (only if protocols differ)
- Per-slave response tracker: `bridge_cam` by ID for multi-master AXI4/AXI5 slaves (or `enable_ooo`), an in-order bridge_id FIFO otherwise

8.6 References

- Jantsch & Tenhunen (eds.), *Networks on Chip*, Kluwer 2003 — the resource-network-interface (RNI) concept that Bridge's master/slave adapters map to: a protocol-dependent wrapper plus a resource-independent crossbar side (ch5 §3.4.2). (Reference archive)
- Eschemann, *The FPGA Programming Handbook*, Packt 2023 — educational secondary source noting that an AXI interconnect's cost scales with active masters, buffering, and whether it is a full crossbar (ch8 §8.3). (Reference archive)

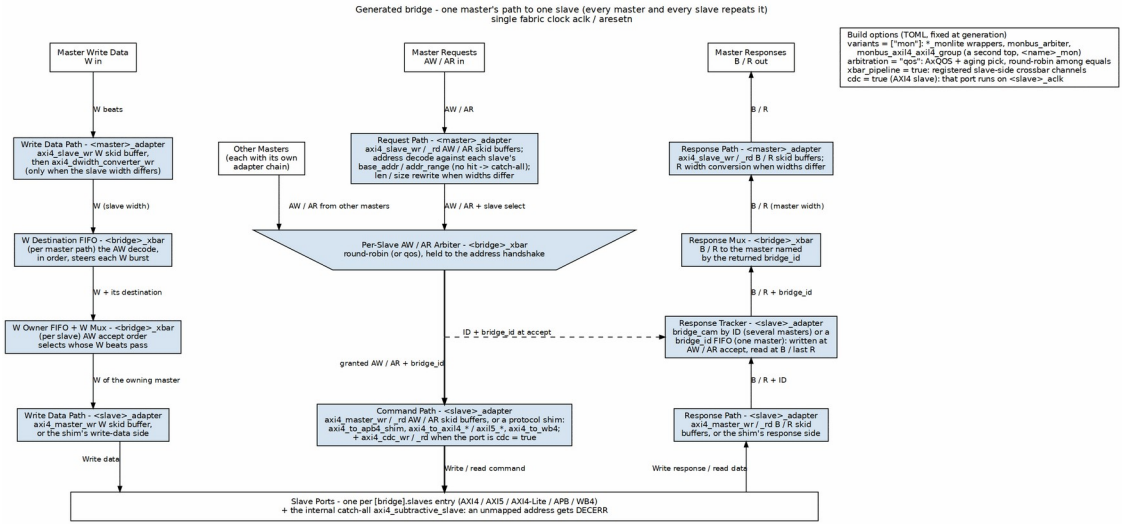
RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

9 Block Diagram

9.1 Bridge Architecture Overview

9.1.1 Figure 3.1: Bridge Block Diagram



Bridge Block Diagram

One master's path through the generated bridge to one slave; every master and every slave repeats it. Write data drops down the left lane, requests take the center through the per-slave arbiter (the trapezium, where every master's AW / AR contend), and B / R responses climb the right lane back to the master named by the returned `bridge_id`. The response tracker in the slave adapter is written at AW / AR accept (dashed) and read when the response returns.

9.2 Functional Blocks

9.2.1 Master Adapter

One per master (`<master>_adapter`), receives transactions from the upstream master:

- **Skid buffers** - `axi4_slave_wr` / `axi4_slave_rd` register every channel at the port
- **Address decode** - a one-hot slave select from each slave's `base_addr` / `addr_range`; no hit selects the catch-all
- **Width conversion** - `axi4_dwidth_converter_wr` / `_rd`, one path per distinct slave width, only where the widths differ
- **Fabric ID** - the AXI ID is prefixed with the master's `BRIDGE_ID` (`{BRIDGE_ID, id}`), so a slave port's ID is wider than a master port's

9.2.2 Crossbar Core

Central switching fabric (`<bridge>_xbar`):

- **5-channel routing** - Independent AW, W, B, AR, R paths
- **Arbitration** - one AW and one AR arbiter per slave, round-robin (or `arbitration = "qos"`), grant held until the address handshake
- **W steering** - a W destination FIFO per master path and a W owner FIFO per slave keep each W burst with its AW
- **Response muxes** - B and R return to the master named by the `bridge_id` the slave adapter hands back

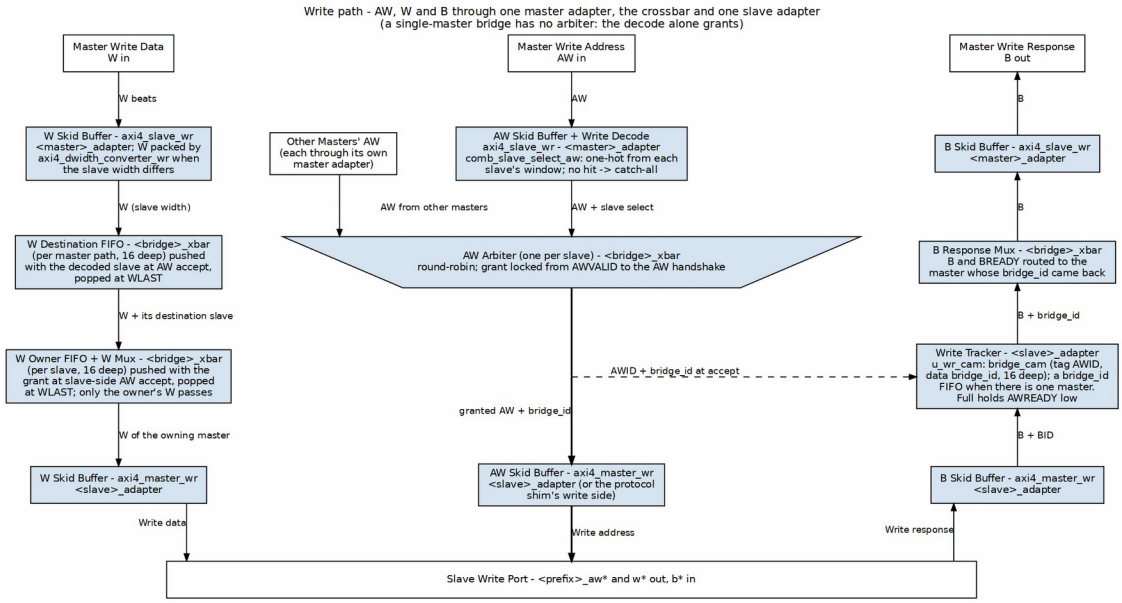
9.2.3 Slave Adapter

One per slave (`<slave>_adapter`), delivers transactions to the downstream slave:

- **Response tracking** - `bridge_cam` keyed by AXI ID when several masters share the slave, or an in-order `bridge_id` FIFO when one master does; written at AW / AR accept, read at B / last R
- **Skid buffers or protocol conversion** - `axi4_master_wr` / `_rd`, or `axi4_to_apb4_shim`, `axi4_to_axil4_*` / `axi4_to_axil5_*`, `axi4_to_wb4`
- **Clock crossing** - `axi4_cdc_wr` / `_rd` when the slave port is `cdc = true`

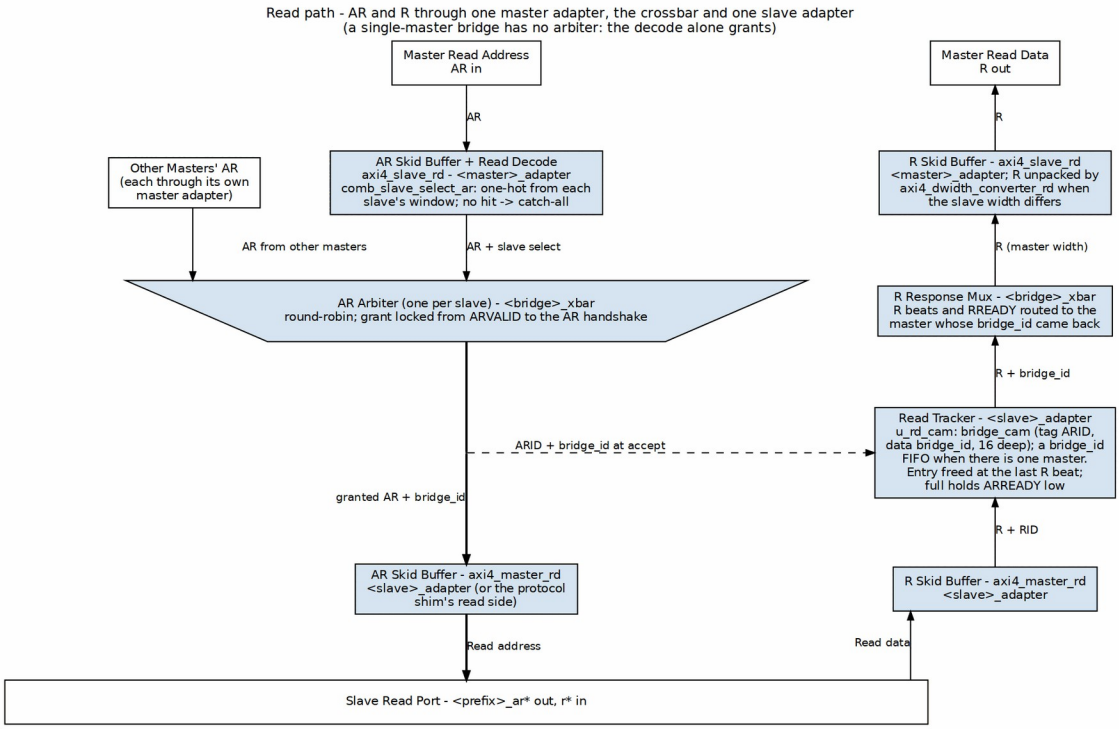
9.3 Data Flow

9.3.1 Figure 3.2: Write Path



Write Path Flow

9.3.2 Figure 3.3: Read Path



Read Path Flow

9.4 Scalability

9.4.1 Topology Parameters

Topology Parameter Scaling

Parameter	Range	Impact
NUM_MASTERS	1-32	Linear MUX growth
NUM_SLAVES	1-256	Linear decoder growth
DATA_WIDTH	32-512	Linear data path growth
ID_WIDTH	1-16	Scales the ID wires (slave side carries {BRIDGE_ID, id}) and the tracker CAM tags

9.4.2 Resource Scaling

- **Logic:** O(M x N) for crossbar core
- **Routing:** O(M + N) for address/response paths
- **Memory:** O(M x outstanding) for ID tracking

9.5 References

- Dally & Towles, *Principles and Practices of Interconnection Networks*, Morgan Kaufmann 2004 — per-slave arbiters on AW and AR as the degenerate case of a separable allocator (ch18 §19.3), and skid buffers as buffered flow control (ch12). (Reference archive)
- Roy, *Advanced Digital System Design*, Springer 2023 — AXI permits W to trail AW independently; the W-owner FIFO records the AW grant so the data path follows the address decision (§18.2.3). (Reference archive)
- Duato/Yalamanchili/Lionel, *Interconnection Networks*, Morgan Kaufmann 2003 — deterministic routing preserves order only along a single path; safe out-of-order completion requires explicit ID-keyed tracking such as the CAM mode Bridge keeps available (ch3/4). (Reference archive)

10 Data Flow

10.1 Transaction Flow Overview

10.1.1 Write Transaction Flow

1. **Master issues AW** - Address and control information
2. **Bridge decodes address** - Determines target slave
3. **Arbitration** - Grants access if multiple masters contend
4. **AW forwarded to slave** - ID unchanged; the originating master is pushed onto the slave adapter's `bridge_id` FIFO
5. **Master sends W data** - Following the granted AW
6. **W forwarded to slave** - Using same grant
7. **Slave returns B response** - with the master's own ID; the FIFO head selects the destination
8. **Bridge routes B to master** - Using ID to find originator

10.1.2 Read Transaction Flow

1. **Master issues AR** - Address and control information
2. **Bridge decodes address** - Determines target slave
3. **Arbitration** - Grants access if multiple masters contend
4. **AR forwarded to slave** - ID unchanged; master recorded in the read `bridge_id` FIFO
5. **Slave returns R data** - Potentially multiple beats
6. **Bridge routes R to master** - Using ID to find originator

10.2 Channel Independence

10.2.1 AXI4 Channel Separation

Each AXI4 channel operates independently:

AXI4 Channel Summary

Channel	Direction	Contents	Arbitration
AW	Master to Slave	Write address	Per-slave
W	Master to Slave	Write data	Follows AW grant
B	Slave to Master	Write response	ID-based routing
AR	Master to Slave	Read address	Per-slave
R	Slave to Master	Read data	ID-based routing

10.2.2 Parallel Operation

- Read and write transactions can proceed simultaneously
- Different masters can access different slaves in parallel
- Same slave serializes access via arbitration

10.3 Bridge ID and ID Widening

10.3.1 Bridge ID (BID) Concept

```
Master ID:                  [MASTER_ID_WIDTH-1:0]
Bridge ID (master index):  BRIDGE_ID_WIDTH = clog2(NUM_MASTERS)
AXI ID on a slave port:    {BRIDGE_ID, master ID}, XBAR_ID_WIDTH bits
```

10.3.2 IDs are widened inside the fabric

With more than one master, each master adapter prepends its index to every request ID (bridge TASK-005), so two masters can never alias an ID at a slave. In `bridge_2x2_rw`:

```
input  logic [3:0]  cpu_m_axi_awid     // master side: 4 bits
output logic [4:0]  ddr_s_axi_awid     // slave side: {BRIDGE_ID, id}
= 5 bits
```

The slave-side width is the widest master's `id_width` plus `$clog2(NUM_MASTERS)` bits (`XBAR_ID_WIDTH` in `<bridge>_pkg.sv`); a slave declared narrower is rejected by the config validator. A single-master bridge prepends nothing. On B and R the master adapter returns only the low master-ID bits, so a master always sees its own ID.

10.4 Response Routing

The prefix bits are not decoded to route a response. Each slave adapter also receives the granted master's index as a sideband (`bridge_id`), stores it at the address handshake, and returns it with the response (`bid_bridge_id` / `rid_bridge_id`); the crossbar's B and R muxes select on that. Which store a slave gets is fixed at generation:

Response Routing

Slave	Tracker	Order the slave may complete in
AXI4 / AXI5 slave, more than one master	`bridge_cam` (`u_wr_cam` / `u_rd_cam`), tag = AXI ID, data = `bridge_id`, 16 deep	any order across IDs
any slave with `enable_ooo = true`	`bridge_cam`	any order across IDs
single-master bridge, apb / axil / wb shim slaves, internal subtractive slave	in-order `bridge_id` FIFO, 16 deep	request order across all IDs

```
// CAM form (multi-master AXI4 slave)
bridge_cam #(.TAG_WIDTH(ID_WIDTH), .DATA_WIDTH(BRIDGE_ID_WIDTH),
             .DEPTH(16), .ALLOW_DUPLICATES(1)) u_wr_cam (
    .allocate_tag  (xbar_ddr_axi_awid),   .allocate_data
(xbar_bridge_id_aw),
    .deallocate_tag(xbar_ddr_axi_bid),    .deallocate_data(bid_bridge_
id), ...);

// FIFO form
wr_fifo[wr_ptr[...]] <= xbar_bridge_id_aw;      // push on AW accept
assign bid_bridge_id  = wr_fifo[rd_ptr[...]];   // route by FIFO head
```

10.4.1 B Channel (Write Response)

1. Slave issues B carrying the fabric ID it was sent
2. The slave adapter looks the BID up in its CAM (or pops its FIFO)
3. The returned `bridge_id` selects the master
4. The master adapter strips the prefix and forwards B

10.4.2 R Channel (Read Data)

1. Slave issues R carrying the fabric ID

2. The slave adapter’s read tracker names the master; the entry is freed on RLAST
3. The master adapter strips the prefix

Either tracker holds 16 transactions; when it is full the slave’s AWREADY / ARREADY toward the crossbar drops (bridge BUG-009).

10.4.3 The requirement the FIFO form creates

A FIFO-tracked slave is routed by *position*, so it must return B/R in request order across ALL IDs. Those slaves are in order by construction (the protocol shims and the subtractive slave) or have a single master; a simulation-only check flags a violation (bridge BUG-008, was BRIDGE-010). A multi-master AXI4/AXI5 slave has no such requirement: the CAM routes by ID.

10.5 Ordering: what the fabric actually guarantees

Per-ID ordering is the slave's AXI obligation; across IDs a CAM-tracked slave may complete in any order. The fabric adds one structural rule:

Per master – **one target at a time.** A master may not have transactions outstanding to more than one slave simultaneously. `aw_gate_ok`/`ar_gate_ok` hold off a new address phase until every outstanding transaction targets the same slave. This is a deadlock fix: the response mux replays in address-issue order, slaves respond in their own order, and cross-slave outstanding transactions from several masters can wedge the heads against each other. Same-slave pipelining is unaffected.

Per FIFO-tracked slave – **responses must come back in order.** See "The requirement the FIFO form creates" above.

The sequence an earlier revision of this page offered as a worked example – master 0 issuing to slave 0 and slave 1 concurrently, then the fast slave answering first – is exactly what `aw_gate_ok` prevents. It cannot occur.

10.6 References

- Dally & Towles, *Principles and Practices of Interconnection Networks*, Morgan Kaufmann 2004 — per-slave arbiters as a separable allocator degenerated to one resource per channel (ch18 §19.3), and skid buffers as buffered flow control (ch12). (Reference archive)
- Roy, *Advanced Digital System Design*, Springer 2023 — AXI's lack of a strict timing relationship between AW and W, and the use of the stored transaction ID to correlate B back to AW (§18.2.3). (Reference archive)
- Duato/Yalamanchili/Lionel, *Interconnection Networks*, Morgan Kaufmann 2003 — deterministic routing preserves order only on a single path; out-of-order responses across IDs need explicit ID-keyed tables such as Bridge's CAM mode (ch3/4). (Reference archive)

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

11 Protocol Support

11.1 Supported Protocols

11.1.1 AXI4 Full

Complete AMBA AXI4 protocol support:

- **5 channels:** AW, W, B, AR, R
- **Burst transactions:** FIXED, INCR, WRAP
- **Transaction IDs:** Configurable width
- **Out-of-order:** NOT supported – responses must return in request order (bridge BUG-008, was BRIDGE-010)
- **Data widths:** 32, 64, 128, 256, 512 bits

11.1.2 AXI4-Lite

Simplified AXI4 for register access:

- **5 channels:** Same as AXI4
- **Single-beat only:** AWLEN/ARLEN = 0
- **No IDs:** Transaction ordering by channel
- **Fixed width:** Typically 32 or 64 bits

11.1.3 AXI3

Legacy AMBA AXI3 protocol support, added as a boundary conversion so the internal crossbar remains AXI4:

- **5 channels:** AW, W, B, AR, R, with `WID` present on write data
- **Burst length:** 1–16 beats on the boundary; longer AXI4 bursts are chopped by the boundary converter
- **Per-port data width:** Same width as the paired AXI4 side; width adaptation happens inside the bridge behind the converter when necessary
- **Lock mapping and interleaving constraints:** set by the converter, not the crossbar — see HAS 6.5 AXI3 Integration

11.1.4 APB (Advanced Peripheral Bus)

Low-power peripheral protocol:

- **Single channel:** Combined address and data

- **Simple handshake:** PSEL, PENABLE, PREADY
- **No bursts:** Single transfer per transaction
- **Low overhead:** Minimal logic for slow peripherals

11.2 Protocol Conversion Matrix

Protocol Conversion Matrix

Master	Slave	Conversion Type	Notes
AXI4	AXI4	Direct	Width conversion if needed
AXI4	AXI4-Lite	Downgrade	Burst split to single beats
AXI4	APB	Full convert	Channel and timing conversion
AXI4-Lite	AXI4	Upgrade	Single-beat pass-through
AXI4-Lite	AXI4-Lite	Direct	Width conversion if needed
AXI4-Lite	APB	Convert	Simplified conversion
AXI5-Lite	any	Upgrade	As AXI4-Lite plus the AXI5-Lite ID; `user` rides the fabric, TRACE/POISON terminate at the boundary (no LOCK/LOOP/MPAM/MECID/NSAID on AXI5-Lite, IHI 0022L

Master	Slave	Conversion Type	Notes
			B2.1.5)
APB / APB5	any	Full convert	`apb{4,5}_to_axi4` front end, then as AXI4-Lite
any	Wishbone B4	Full convert	`axi4_to_wb4`: one transfer per beat, in-order termination
Wishbone B4	any	Full convert	`wb4_to_axi4` front end, then as AXI4-Lite
AXI4	AXI3	Protocol convert	`axi4_to_axi3` slave boundary; same data width only
AXI3	AXI4	Protocol convert	`axi3_to_axi4` master boundary; same data width only

11.3 Automatic Conversion Shims at Slave Boundary

When a slave port's TOML entry names a protocol other than native AXI4 (e.g., `protocol = "axi3"`, `protocol = "axil"` or `protocol = "apb"`), the generator emits the conversion shims between the crossbar core and that slave port. This happens once, at generation time; there is nothing to configure at runtime.

Supported Slave Protocols – these are the exact TOML values the generator accepts (`config_validator.valid_protocols`); anything else is a generation error, so spelling matters:

Slave-port protocol values

`protocol`	Meaning	Shim emitted at the slave boundary
`axi4`	Native AXI4	none
`axi3`	AMBA AXI3	`axi4_to_axi3`
`axi5`	AXI5, interop mode	`axi5_master_{wr,rd}` boundary wrappers
`axil`	AXI4-Lite	`axi4_to_axil4_{rd,wr}`
`axil5`	AXI5-Lite	`axi4_to_axil5_{rd,wr}`
`apb`	APB3/APB4	`axi4_to_apb4_shim`
`apb5`	APB5	`axi4_to_apb5_shim`
`wb4`	Wishbone B4 (pipelined)	`axi4_to_wb4`

Note the spelling: it is `axil`, not `axi4lite`. Earlier revisions of this page used the latter, which the generator rejects.

The shim modules live in `projects/components/utility-ip/converters/rtl/`; the generator instantiates them into each generated top-level module. The slave port still presents the configured protocol (AXIL or APB) to the outside; inside, the crossbar core is uniformly AXI4.

11.4 Automatic Conversion at the Master Boundary

The same `protocol` values are legal on a master port (bridge TASK-004, was BRIDGE-014). The bridge presents the requester's own protocol on the boundary – the AXI4-Lite signal set, the AXI5-Lite set with its sideband, or the APB completer set – and the master adapter converts to the AXI4 the crossbar speaks:

Master-port protocol values

`protocol`	Boundary presents	Conversion in the master adapter
`axi4`	AXI4	none (timing wrapper only)
`axi3`	AXI3	`axi3_to_axi4`, then the AXI4 timing wrapper
`axi5`	AXI5 (minus REGION, plus enabled features)	`axi5_slave_{wr,rd}` boundary wrappers
`axil`	AXI4-Lite	AXI4 extras tied (`AxLEN=0`, INCR, `WLAST=1`, ID = placeholder); the wide-slave aligner toward wider slaves
`axil5`	AXI5-Lite: ID + optional signals	as `axil`, but `awid`/`arid` -> the AXI4 ID and `bid`/`rid` back; `user` -> the 1-bit USER fields; TRACE/POISON terminated at the top
`apb`	APB4 completer	`apb4_to_axi4`, then the AXI4 timing wrapper
`apb5`	APB5 completer (+ `PAUSER/PWUSER/PWAKEUP` in – all requester-driven, IHI 0024E 3.7.2 – `PRUSER/PBUSER` out)	`apb5_to_axi4` (`PAUSER[0]`/`PWUSER[0]` -> USER), then the AXI4 timing wrapper
`wb4`	Wishbone B4 completer (pipelined)	`wb4_to_axi4`, then the AXI4 timing wrapper; SLVERR/DECERR

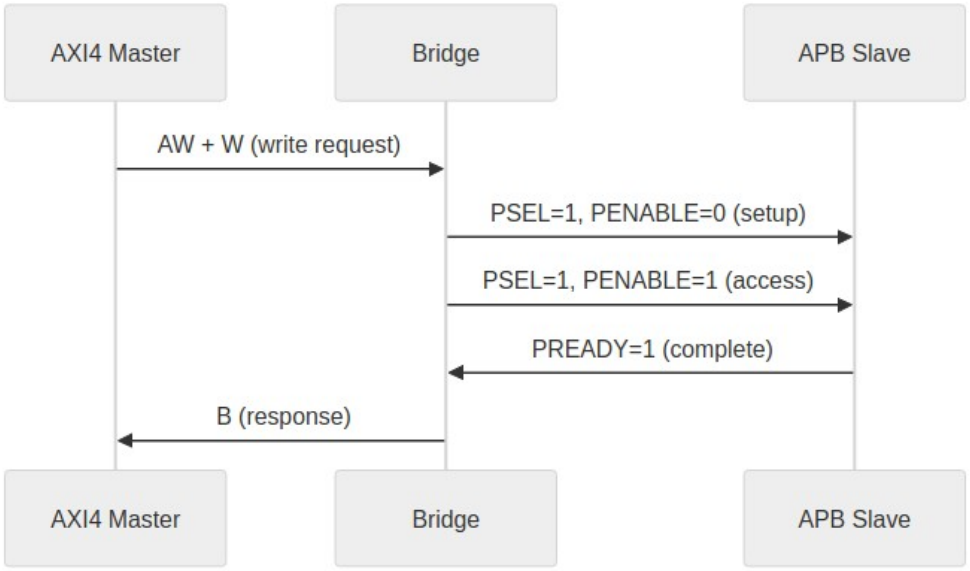
protocol	Boundary presents	Conversion in the master adapter
		terminate ERR

Rules the validator enforces on these ports: AXI4-Lite masters have `id_width = 0` (no ID pins exist; the fabric ID is the master index, bridge TASK-005 (was BRIDGE-016)), AXI5-Lite masters `id_width >= 1` (AWID/ARID are required on an AXI5-Lite Subordinate, IHI 0022L B2.1.5, GitHub #102 item 3), APB and Wishbone masters have `addr_width = 32` (the requester addresses the whole fabric – an APB *slave* port's `PADDR` is a window offset, a master's is not), and `axi5_features` on an `axil5` port may name only `user`, the one group with an AXI4 destination (`exclusive`, `loop`, `mpam`, `mecid` and `nsaid` are rejected as not existing on AXI5-Lite).

11.5 AXI4 to APB Conversion

11.5.1 Conversion Process

11.5.2 Figure 3.4: AXI4 to APB Conversion Sequence



AXI4 to APB Conversion

11.5.3 Burst Handling

- AXI4 bursts split into individual APB transfers
- WSTRB converted to per-byte enables
- B response generated after all beats complete

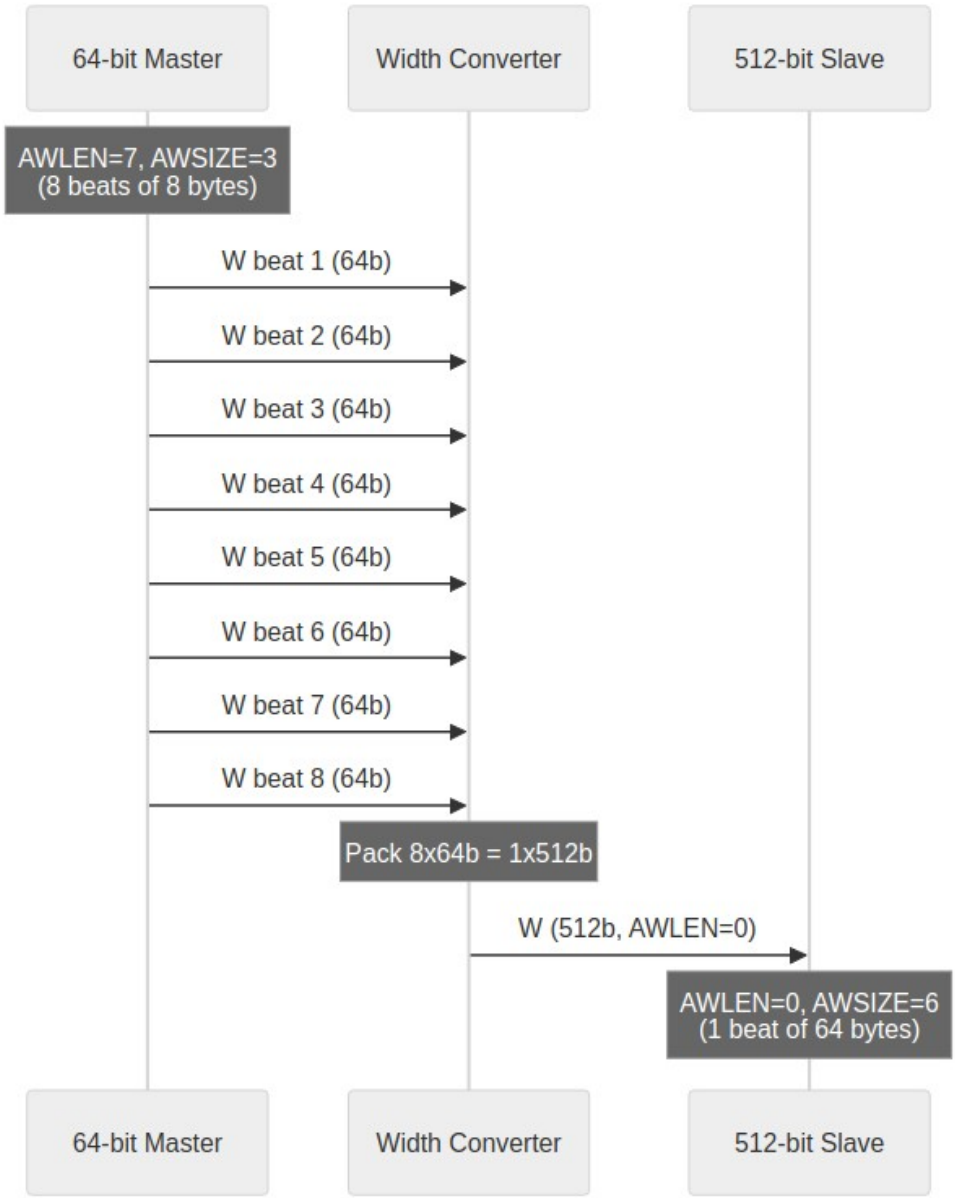
11.5.4 Timing Considerations

- APB is inherently slower (minimum 2 cycles per transfer)
- Pipeline stalls during APB access
- Consider separate APB crossbar for multiple slow peripherals

11.6 Width Conversion

11.6.1 Upsize (Narrow to Wide)

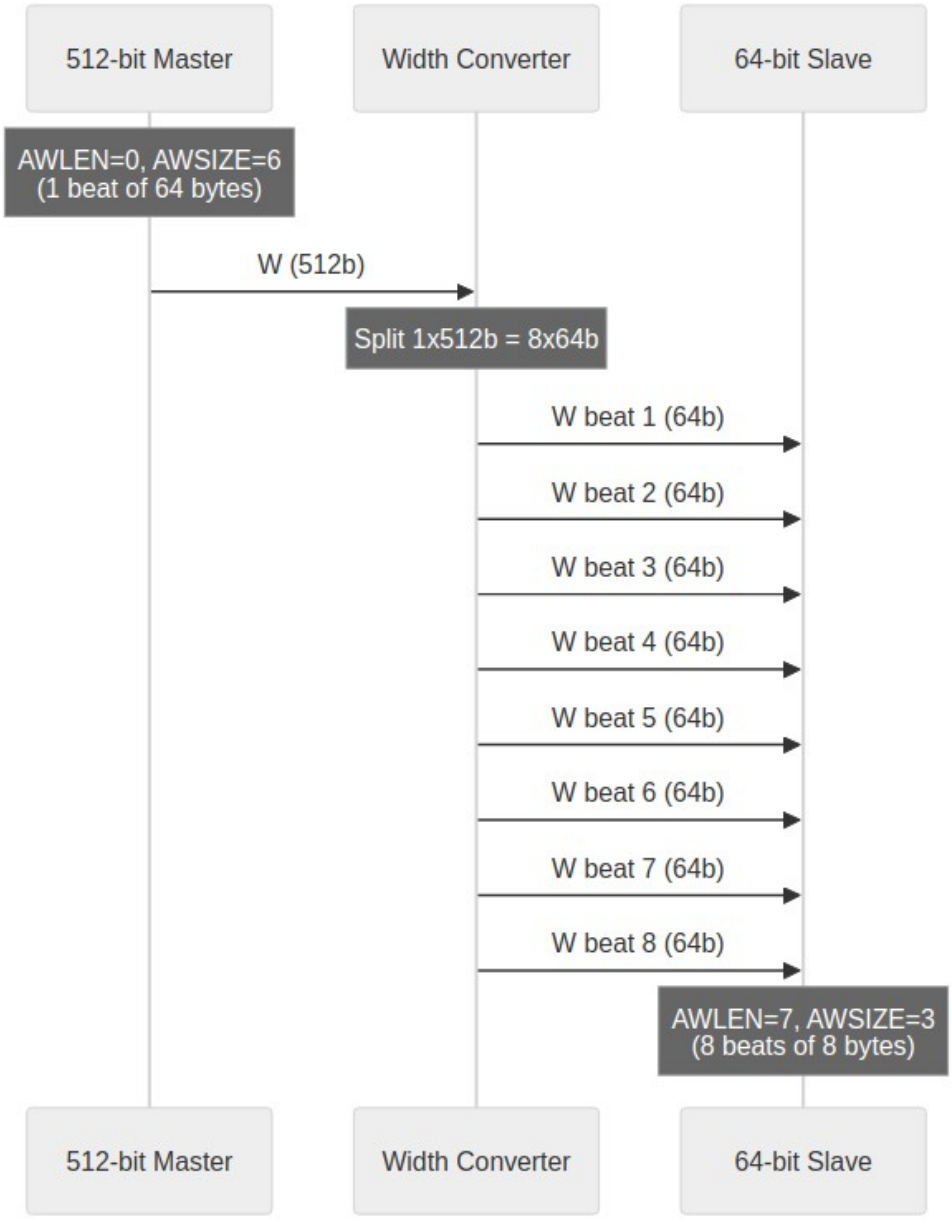
11.6.2 Figure 3.5: Width Upsize Conversion



Width Upsize

11.6.3 Downsize (Wide to Narrow)

11.6.4 Figure 3.6: Width Downsize Conversion



Width Downsize

11.7 Channel-Specific Protocol

11.7.1 Write-Only Masters

Only AW, W, B channels active:

- AR permanently deasserted
- R channel ignored (no routing logic)
- Reduced signal count and logic

11.7.2 Read-Only Masters

Only AR, R channels active:

- AW, W, B channels ignored
- No write arbitration for this master
- Reduced signal count and logic

11.8 ACE and Coherency — Explicitly Out of Scope

The bridge implements AMBA 5 **AXI5** interop and sideband features, but it is **not a coherence interconnect**: there are no ACE snoop channels (AC/CR/CD), no `ACVALID/ACREADY/CRVALID/CRREADY/CDVALID/CDREADY` signals anywhere in the generated RTL, and cache-coherent masters are not supported. Coherent systems belong to the cache-ip domain (see the amber/jet/onxy work). This mirrors `PRD.md`: "No ACE protocol extensions (cache coherency)."

11.9 References

- Eschemann, *The FPGA Programming Handbook*, Packt 2023 — educational secondary source for AXI4 full and AXI4-Lite channel structure and handshake rules (ch8 §8.3). (Reference archive)
- Roy, *Advanced Digital System Design*, Springer 2023 — secondary source for AXI channel structure and headline protocol features (§18.2.2, §18.2.2.1). (Reference archive)
- Maaref, *Architecting and Building High-Speed SoCs*, Packt 2022 — AMBA-suite context and the evolution of AXI4 features such as QoS/Region (ch3 §4.2). (Reference archive)
- ARM, *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022E — the authoritative AXI4 protocol definition. (Reference archive)
- ARM, *AMBA 3 AXI Protocol Specification*, ARM IHI 0022 — the AXI3 protocol authority; 4-bit `AxLEN`, 2-bit `AxLOCK`, write-data `WID`, absence of `QoS`/`REGION`. (Reference archive)

---

12 AXI4 Interface Overview

12.1 Interface Organization

12.1.1 Master-Side Interfaces

Bridge presents **slave interfaces** to upstream masters. Each master port includes:

Master-Side Interface Channels

Channel	Signals	Direction (to Bridge)
AW	AWVALID, AWREADY, AWADDR, AWID, AWLEN, AWSIZE, AWBURST, ...	Input
W	WVALID, WREADY, WDATA, WSTRB, WLAST	Input
B	BVALID, BREADY, BID, BRESP	Output
AR	ARVALID, ARREADY, ARADDR, ARID, ARLEN, ARSIZE, ARBURST, ...	Input
R	RVALID, RREADY, RDATA, RID, RRESP, RLAST	Output

12.1.2 Slave-Side Interfaces

Bridge presents **master interfaces** to downstream slaves. Each slave port includes:

Slave-Side Interface Channels

Channel	Signals	Direction (from Bridge)
AW	AWVALID, AWREADY, AWADDR, AWID, AWLEN, AWSIZE, AWBURST, …	Output
W	WVALID, WREADY, WDATA, WSTRB, WLAST	Output
B	BVALID, BREADY, BID, BRESP	Input
AR	ARVALID, ARREADY, ARADDR, ARID, ARLEN, ARSIZE, ARBURST, …	Output
R	RVALID, RREADY, RDATA, RID, RRESP, RLAST	Input

12.2 Signal Naming Convention

12.2.1 Custom Prefixes

Each port uses a custom prefix for signal naming:

```
// Example: Master 0 with prefix "cpu_m_axi_"
input  logic [31:0] cpu_m_axi_awaddr,
input  logic [3:0]  cpu_m_axi_awid,
input  logic        cpu_m_axi_awvalid,
output logic        cpu_m_axi_awready,
// ... remaining AW signals

// Example: Slave 0 with prefix "ddr_s_axi_"
output logic [31:0] ddr_s_axi_awaddr,
output logic [4:0]  ddr_s_axi_awid,  // {BRIDGE_ID, id}: master width
+ clog2(NUM_MASTERS)
output logic        ddr_s_axi_awvalid,
input  logic        ddr_s_axi_awready,
```

12.2.2 ID Width Extension

With more than one master, slave-side IDs are WIDER than master-side:

```
Master ID Width: id_width of the master port (widest master =
MASTER_ID_WIDTH)
Bridge ID Width: clog2(NUM_MASTERS)           (ID_PREFIX_WIDTH; 0 for
one master)
Slave ID Width:  MASTER_ID_WIDTH + ID_PREFIX_WIDTH  (XBAR_ID_WIDTH)

bridge_2x2_rw: cpu_m_axi_awid [3:0]  ->  ddr_s_axi_awid [4:0]
```

Each master adapter sends `{BRIDGE_ID, id}` into the fabric and strips the prefix from B and R. A slave port's declared `id_width` must be at least `XBAR_ID_WIDTH`; the validator rejects a narrower AXI3/AXI4/AXI5 slave. Responses are routed by the `bridge_id` sideband the slave adapter stores: a `bridge_cam` looked up by BID/RID for a multi-master AXI4/AXI5 slave (any completion order), or an in-order FIFO for single-master bridges and shim slaves (request order required, bridge BUG-008). See HAS 3.2.

12.3 Channel-Specific Interfaces

12.3.1 Write-Only Master (wr)

Only AW, W, B channels generated:

```
// Write Address Channel
input  logic [ADDR_WIDTH-1:0] descr_m_axi_awaddr,
input  logic [ID_WIDTH-1:0]   descr_m_axi_awid,
// ... full AW channel

// Write Data Channel
input  logic [DATA_WIDTH-1:0] descr_m_axi_wdata,
input  logic [STRB_WIDTH-1:0] descr_m_axi_wstrb,
// ... full W channel

// Write Response Channel
output logic [ID_WIDTH-1:0]   descr_m_axi_bid,
output logic [1:0]            descr_m_axi_bresp,
// ... full B channel

// NO AR or R channels
```

12.3.2 Read-Only Master (rd)

Only AR, R channels generated:

```
// Read Address Channel
input  logic [ADDR_WIDTH-1:0] src_m_axi_araddr,
input  logic [ID_WIDTH-1:0]   src_m_axi_arid,
// ... full AR channel

// Read Data Channel
output logic [DATA_WIDTH-1:0] src_m_axi_rdata,
output logic [ID_WIDTH-1:0]   src_m_axi_rid,
// ... full R channel

// NO AW, W, or B channels
```

12.4 Monitor Bus Output (when `variants` includes `mon`)

When monitor collection is enabled (via the bridge TOML `variants` list including `"mon"`), each bridge instance exposes a unified monitor interface at the bridge top level:

Monitor Bus Output

Signal	Type	Width	Direction	Description
`s_mon_axil_*`	AXIL Slave	32-bit addr, 64-bit data	Input	Monitor packet read interface for CPU consumption
`m_axil_mon_*`	AXIL Master	32-bit addr, 64-bit data	Output	Monitor packet write interface for bulk trace capture
`stream_irq`	logic	1-bit	Output	Interrupt signal (asserted when error FIFO has records)

Per-port collection comes from the `axi4_master_{rd,wr}_monlite` and `axi4_slave_{rd,wr}_monlite` wrappers (`axi_monitor_lite`; the full `_mon` wrappers are no longer used in generated bridges, 2026-09-26).

Packet Format: the canonical 128-bit packet layout and field definitions live in `docs/markdown/rtl-amba/includes/ocelus_package_spec.md` — see that reference for bit-field descriptions, protocol-type enumerations, and packet-type codes.

12.5 Handshake Protocol

12.5.1 Valid/Ready Handshake

All channels use AXI4 valid/ready handshake:

```
Transfer occurs when: VALID && READY
```

12.5.2 Backpressure

- Bridge asserts READY when able to accept
- Masters must hold VALID until READY
- Bridge does not drop transactions

12.6 Burst Support

12.6.1 Supported Burst Types

Supported Burst Types

AWBURST/ARBURST	Type	Description
2'b00	FIXED	Same address each beat
2'b01	INCR	Incrementing address
2'b10	WRAP	Wrapping at boundary
2'b11	Reserved	Not used

12.6.2 Burst Length

- AWLEN/ARLEN: 8-bit field
- Length = AXLEN + 1 (1 to 256 beats)
- Burst does not cross 4KB boundary (AXI4 rule)

13 APB Interface

13.1 Overview

APB is where the slow things live: UARTs, GPIO, configuration registers. This page covers the bridge's APB surface — the signal set, the port-naming convention, transaction timing, and what the AXI4-to-APB conversion actually does.

13.2 Ports

13.2.1 Signal Definition

APB Signal Definitions

Signal	Width	Direction	Description
PSEL	1	Output	Slave select
PENABLE	1	Output	Transaction phase
PADDR	ADDR_WIDTH	Output	Address
PWRITE	1	Output	Write enable
PWDATA	DATA_WIDTH	Output	Write data
PSTRB	DATA_WIDTH/8	Output	Byte strobes
PPROT	3	Output	Protection
PRDATA	DATA_WIDTH	Input	Read data
PSLVERR	1	Input	Slave error
PREADY	1	Input	Slave ready

13.2.2 Signal Naming

APB slave ports use custom prefixes:

```
// Example: APB slave with prefix "uart_apb_"
output logic        uart_apb_psel,
output logic        uart_apb_penable,
output logic [31:0] uart_apb_paddr,
output logic        uart_apb_pwrite,
output logic [31:0] uart_apb_pwdata,
output logic [3:0]  uart_apb_pstrb,
output logic [2:0]  uart_apb_pprot,
input  logic [31:0] uart_apb_prdata,
input  logic        uart_apb_pslverr,
input  logic        uart_apb_pready
```

13.3 Functional Description

13.3.1 AXI4 to APB Conversion

When an AXI4 master accesses an APB slave, the bridge:

1. **Burst splitting** - AXI4 bursts become multiple APB transfers
2. **Channel mapping** - AW+W combined into PADDR+PWDATA
3. **Response generation** - APB PSLVERR maps to AXI4 BRESP/RRESP
4. **Timing adaptation** - Insert wait states as needed

13.3.2 Burst Handling

AXI4 to APB Burst Conversion

AXI4 AWLEN	APB Transfers
0 (1 beat)	1 transfer
1 (2 beats)	2 transfers
N (N+1 beats)	N+1 transfers

13.3.3 Error Mapping

APB to AXI4 Error Mapping

APB PSLVERR	AXI4 BRESP/RRESP
0 (OK)	2'b00 (OKAY)
1 (Error)	2'b10 (SLVERR)

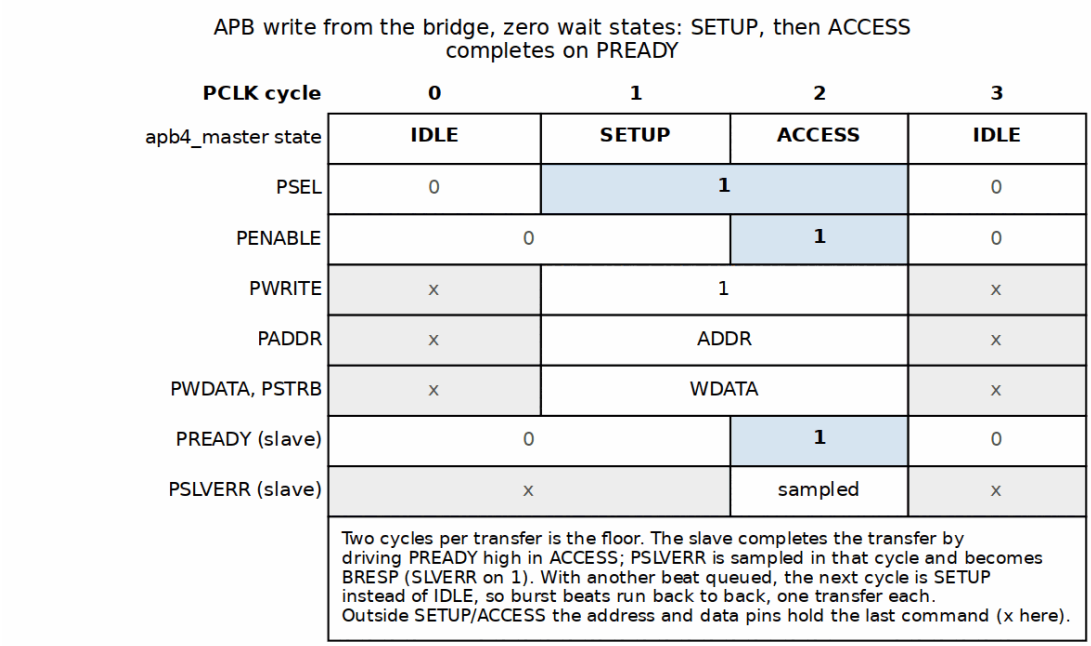
13.4 Timing

Every APB transfer pays for a setup phase plus an access phase, so the floor is two cycles per transfer. There is no pipelining — transfers run sequentially — and no bursts: each beat requires the full handshake.

13.5 Waveforms

13.5.1 Write Transaction

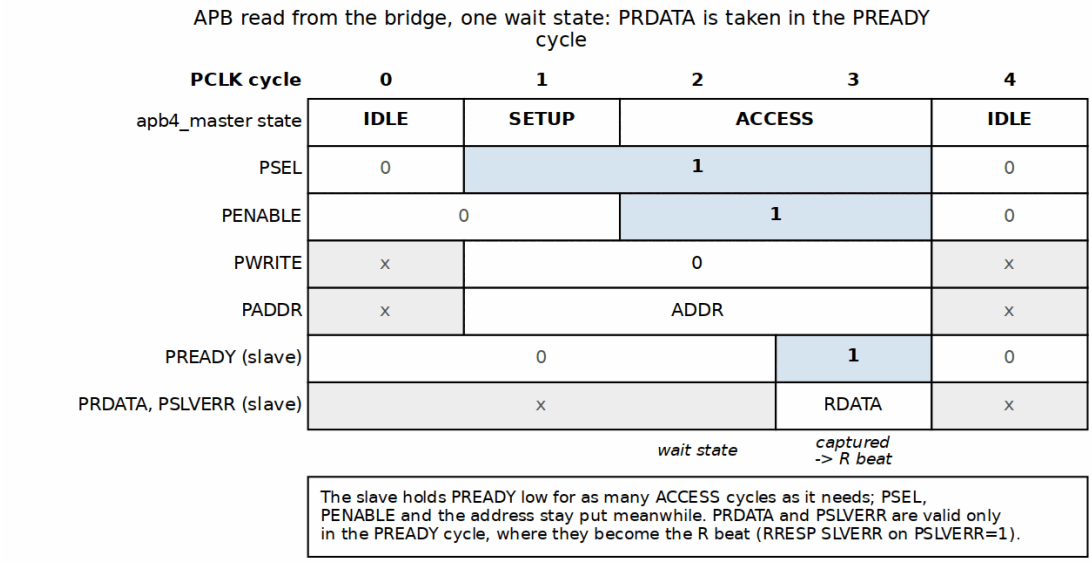
13.5.1.1 Figure 4.1: APB Write Transfer (Zero Wait States)



APB Write Transfer (Zero Wait States)

13.5.2 Read Transaction

13.5.2.1 Figure 4.2: APB Read Transfer (One Wait State)



APB Read Transfer (One Wait State)

13.6 APB Requester Ports

Since bridge TASK-004 (was BRIDGE-014) an APB port can also be a *master*: `protocol = "apb"` or `"apb5"` in `[[bridge.masters]]`. The signal set is the same ten (fifteen for APB5) with the directions of the slave table reversed – the bridge is the completer, so `PSEL`, `PENABLE`, `PADDR[31:0]`, `PWRITE`, `PWDATA`, `PSTRB`, `PPROT` are inputs and `PREADY`, `PRDATA`, `PSLVERR` outputs. `PADDR` is the full 32-bit fabric address (the validator insists on `addr_width = 32`); each transfer becomes one single-beat AXI4 transaction through `apb4_to_axi4` / `apb5_to_axi4`, and both `SLVERR` and `DECERR` come back as `PSLVERR`. The requester is one-outstanding by nature of APB, so throughput is one transfer per fabric round trip.

13.7 Design Notes

- Use APB only for slow peripherals (UART, GPIO, config registers)
- Group APB slaves behind dedicated sub-crossbar
- Use AXI4/AXI4-Lite for higher-bandwidth slaves

---

14 Clock and Reset

14.1 Overview

One clock, one reset. The AXI fabric runs in a single synchronous domain — there is exactly one exception, an AXI4 slave port declared `cdc = true`, and it gets its own discussion below — and the reset is asynchronous, active-low. This page covers what that means for your clock tree, your reset strategy, and your constraint file.

14.2 Ports

14.2.1 Clock

Clock Requirements

Signal	Description	Requirements
aclk	System clock	All interfaces synchronous
Frequency	Operating frequency	Design-dependent
Duty cycle	Clock duty cycle	40-60% typical

14.2.2 Reset

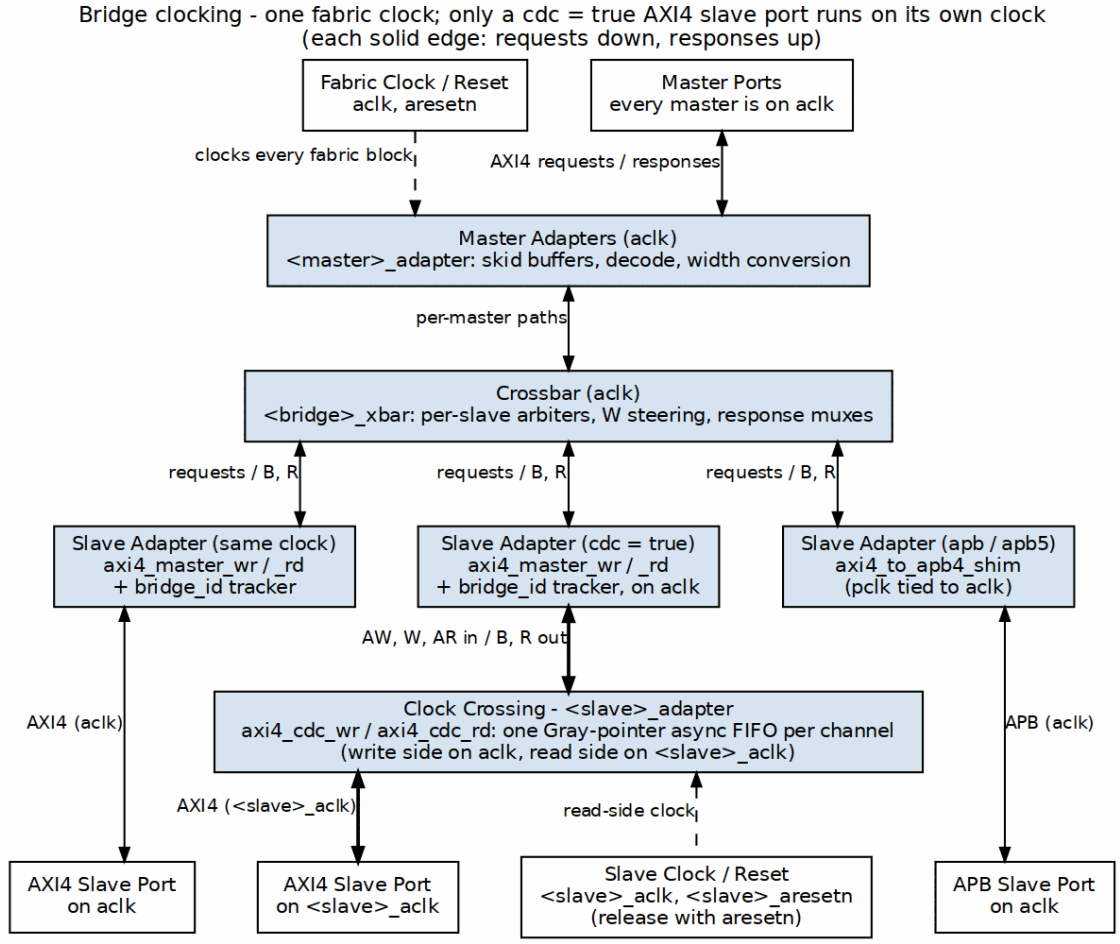
Reset Signal

Signal	Description	Polarity
aresetn	Async reset	Active-low

14.3 Functional Description

14.3.1 One Fabric Clock

14.3.1.1 Figure 4.3: Clock Distribution



Clock Distribution

Every master port and the fabric itself run on `aclk`. A slave port may run on its own clock when declared `cdc = true` (next section); every other slave port shares `aclk` too.

14.3.2 Clock Domain Crossing: CDC Slave Ports

The FABRIC is one clock domain (`aclk`). Since bridge TASK-006 (was BRIDGE-017) a slave port can live in another: `cdc = true` on an AXI4 slave adds `<slave>_aclk` / `<slave>_aresetn` to the bridge top, and the slave adapter carries AW, W and AR to the port and B, R back through `axi4_cdc_{wr,rd}` – one Gray-pointer async FIFO per channel (`rtl/amba/axi4/axi4_cdc.md`) – downstream of its timing wrapper, which stays on `aclk` with its monitor and the bridge-id

tracking. Everything upstream of the crossing is unchanged; the port itself runs entirely on its own clock, at one beat per cycle of the slower domain.

CDC slave port clock pins (`cdc = true`)

Signal	Direction	Description
`<slave>_aclk`	Input	the CDC slave port's clock
`<slave>_aresetn`	Input	its active-low reset; release together with `aresetn` (a one-sided reset is not safe on the async FIFOs – handbook `design/cdc.md`)

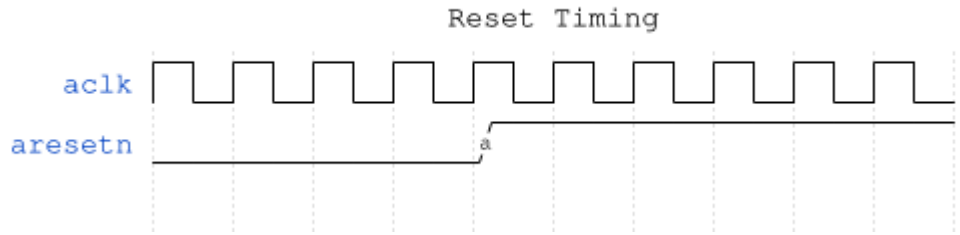
Limits: slave ports only, `protocol = "axi4"` (the crossing carries the AXI4 signal set; AXI5 sideband and the APB/AXIL/WB4 shims have no crossing yet – the validator rejects those). Measured on `bridge_2x2_rw_cdc` (`test_bridge_2x2_rw_cdc_ratio`) with the slave clock at 3, 10 and 23 ns against a 10 ns fabric: every burst from both masters completes, the port's beats appear on the slave clock, the sustained rate is set by the slower clock (>= 0.85 beat per cycle of it), and the same-clock port beside it is unaffected.

An `apb`/`apb5` slave's `axi4_to_apb4_shim` core is built around two async FIFOs and has its own `pclk`/`presetn` pins, but the generator ties them to `aclk`/`aresetn` (`periph_adapter.sv` in `bridge_2x3_apb_req`), and the bridge top has no APB clock port. A generated APB slave port therefore runs on `aclk`. For every port that is not `cdc = true`:

- All inputs sampled on aclk rising edge
- All outputs generated on aclk rising edge
- External CDC required if a master, or a non-AXI4 slave, uses a different clock

14.3.3 Reset Behavior

14.3.3.1 Figure 4.4: Reset Timing



Reset Timing

Reset is active-low: the bridge is in reset whenever `aresetn = 0`.

14.3.4 Reset Effects

During reset (aresetn = 0):

Reset Effects

Component	State
Arbiters	Cleared (no grants)
bridge_id FIFOs	Pointers cleared (no outstanding)
FIFOs	Emptied
Outputs	Deasserted (VALID = 0)

14.3.5 Outstanding Transactions

Warning: reset discards transactions in flight:

- Master may see timeout (no response)
- Slave may receive partial transaction
- System must ensure quiescence before reset

14.3.6 Reset Release

After reset release (aresetn = 1):

1. Bridge accepts new transactions after 1 cycle
2. All state machines start in IDLE
3. No residual grants or locks

14.4 Timing

14.4.1 Setup and Hold

All inputs must meet setup/hold relative to aclk:

Timing Constraints

Constraint	Typical Value
Setup time	Process-dependent
Hold time	Process-dependent

14.4.2 Clock-to-Output

All outputs appear after aclk rising edge:

Clock-to-Output Delays

Path	Typical Delay
aclk to VALID	1 cycle (registered)
aclk to READY	1 cycle (registered)
aclk to DATA	1 cycle (registered)

READY is registered like the rest, which an earlier revision of this table gave as “Combinational”. Every external port terminates in a `gaxi_skid_buffer` inside its boundary wrapper – a master port’s `awready` is `axi4_slave_wr.s_axi_awready`, wired straight to the skid’s `wr_ready`, and that is assigned in an `ALWAYS_FF_RST` block. A master budgeting a combinational ready path against this bridge would be wrong by a register stage.

14.5 Usage Example

14.5.1 Recommended Constraints

```
# Example SDC constraints
create_clock -period 10 [get_ports aclk]  ;# 100 MHz

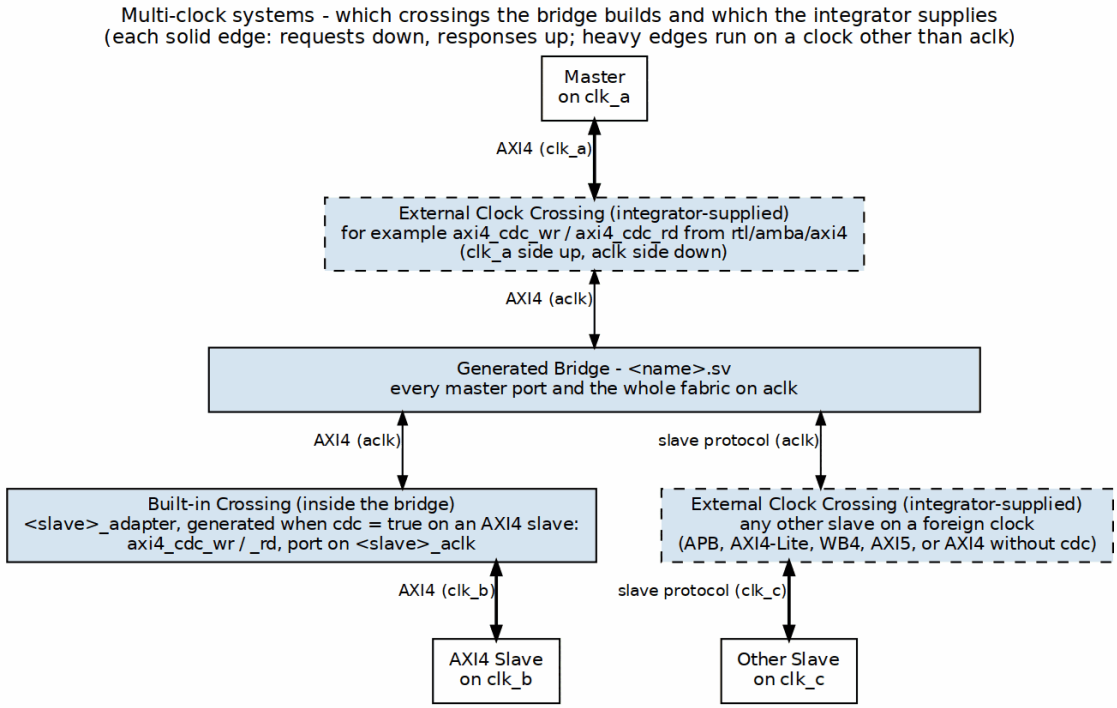
# Reset is async - synchronize externally
set_false_path -from [get_ports aresetn]
```

14.6 Design Notes

14.6.1 External CDC Required

For master ports, and for slave ports that are not `cdc = true` AXI4 ports, in systems with multiple clock domains:

14.6.1.1 Figure 4.5: Multi-Clock CDC



Multi-Clock CDC

14.6.2 CDC Recommendations

- Use AXI4 async FIFO bridges
- Consider AXI4 clock converter IP
- Ensure proper gray-coding for pointers

15 AXI5 and APB5 Interfaces (AMBA5 Support)

15.1 Overview

The bridge is AXI4-shaped internally with every AXI5 feature the library wrappers carry riding alongside in the channel structs — since bridge TASK-007 (was BRIDGE-018) that includes Memory Tagging and read-data chunking, so there is no AXI5 signal an `axi5` port can present that the fabric cannot carry natively to another `axi5` port. Any master or slave port can be declared AMBA5. This chapter covers the external surfaces; the mechanism lives in the MAS (AMBA5 Boundary and Native Sideband).

15.2 Parameters

15.2.1 Declaring AMBA5 Ports

AMBA5 is declared per port, in the TOML:

```
[[bridge.masters]]
name = "cpu_wr"
protocol = "axi5"
axi5_features = ["trace", "atomic"]   # optional; empty = base AXI5
# ... standard fields unchanged

[[bridge.slaves]]
name = "periph5"
protocol = "apb5"                     # APB5 peripheral via the apb5
shim
channels = "rw"                       # APB rules unchanged (rw-only,
32-bit)

[[bridge.masters]]
name = "lite5"
protocol = "axil5"                    # AXI5-Lite requester (bridge
TASK-004, was BRIDGE-014)
id_width = 1                          # AWID/ARID: required on an
AXI5-Lite Subordinate
axi5_features = ["user"]              # the one group with an AXI4
destination

[[bridge.masters]]
name = "apb5m"
protocol = "apb5"                     # APB5 requester (bridge TASK-
004, was BRIDGE-014)
id_width = 0
addr_width = 32                       # the requester addresses the
whole fabric
channels = "rw"
```

15.3 Ports

15.3.1 AXI5 Port Surface

An `axi5` port exposes the AXI4 signal set **minus AW/ARREGION** (AXI5 removed REGION) **plus** the enabled features' sideband signals. Disabled features' signals are not exposed at all — the boundary wrapper ties them off internally.

AXI5 Port Surface

Feature	Signals added	Class
`nsaid`	`aw/arnsaid[3:0]`	Droppable sideband
`trace`	`aw/artrace`, `btrace`, `rtrace`	Droppable sideband
`mpam`	`aw/armpam[10:0]`	Droppable sideband
`mecid`	`aw/armecid[15:0]`	Droppable sideband
`unique`	`aw/aridunq` in; `bidunq`, `ridunq` out (IHI 0022L A5.2)	Droppable sideband; the response indicators are regenerated at the master port (below)
`wakeup`	`awakeup` (one signal for both request channels, present on rd-only, wr-only and rw ports alike)	Not path sideband; see "AWAKEUP" below
`poison`	`wpoison`, `rpoison`	**Connectivity-gated**
`atomic`	`awatop[5:0]`	**Connectivity-gated**; read-return classes native on rw ports, DECERR on write-only ports
`mte`	`aw/artagop[1:0]`, `wtag`, `wtagupdate`, `btagmatch[1:0]`, `rtag` (tag buses are 4 bits per 16 bytes of data, `wtagupdate` one bit per 16 bytes; IHI 0022L A12.2 has no `AWTAG`, `BTAG` or	**Connectivity-gated**; 128-bit ports or wider (bridge TASK-007, was BRIDGE-018)

Feature	Signals added	Class
	RTAGMATCH, and BTAGMATCH encodes Table A12.14: 00 none, 01 separate, 10 fail, 11 pass)	
chunking	archunken, rchunkv, rchunknum[3:0], rchunkstrb (one bit per 128 bits of data)	Droppable sideband; 128-bit ports or wider (bridge TASK-007, was BRIDGE-018)

15.3.2 APB5 Slave Surface

An apb5 slave exposes the APB4 requester surface plus the APB5 sideband, mirroring rtl/amba/apb5/apb5_slave.sv pin-for-pin:

APB5 Slave Surface

Direction	Signals
Requester → completer	APB4 set + PAUSER, PWUSER (driven '0 — nothing upstream sources them) + PWAKEUP (below)
Completer → requester	APB4 set + PRUSER, PBUSER (accepted and terminated)

PWAKEUP is a **requester output** (IHI 0024E 3.7.2, Table 3-7). The bridge port used to take it as an input, following a library convention that had the direction backwards (GitHub #102 item 7). axi4_to_apb5_shim now drives it from a flop of PSEL: glitch-free, high from the cycle after the Setup phase and held through the PREADY cycle, so a completer that waits for PWAKEUP before PREADY always receives it, and low when no transfer is in progress. 3.7.2 permits assertion during or after PSEL; the recommended one-cycle lead is not available because the command queue that would predict a transfer is inside the shared APB4 conversion core. No *CHK check signals are exposed (the port is Check_Type = False).

The transfer protocol is unchanged from APB4, so the axi4_to_apb5_shim is a sideband wrapper over the APB4 conversion core; APB constraints (rw-only, 32-bit data) apply unchanged.

15.3.3 AXI5-Lite Master Surface

An axil5 master port (bridge TASK-004, was BRIDGE-014) exposes the AXI4-Lite requester set, the AXI5-Lite ID (awid/arid in, bid/rid out, id_width bits – required on an AXI5-Lite Subordinate, IHI 0022L B2.1.5, so id_width must be at least 1), and the **whole** AXI5-Lite optional set, every group whether or not it is enabled – the same rule as the slave surface, so the boundary keeps one

shape. The directions flip relative to the slave table: requester-driven groups are bridge inputs, completer-driven groups bridge outputs.

AXI5-Lite sideband groups at an `axil5` master boundary

Group	Signals	Behaviour
ID	`awid`, `arid` in; `bid`, `rid` out	reach the adapter's AXI4 face, ride the fabric inside `{master index, id}` and come back unchanged
FORWARDED (when enabled)	`awuser`, `wuser`, `aruser` (`user`)	reach the adapter's AXI4 face and ride the fabric to the slave
TERMINATED inputs	`aw/artrace`, `wpoison`; `awuser`/… when `user` is off	consumed at the bridge top (a reduction into an `_unused` wire), never floating
TERMINATED outputs	`btrace`, `rtrace`, `rpoison`; `buser`/`ruser` when `user` is off	driven `'0`
Response USER	`buser`, `ruser` with `user` on	connected, but read 0: the master adapters tie response-side USER (see the PRD)

15.3.4 APB5 Master Surface

An `apb5` master port makes the bridge the APB5 **completer**: the APB4 set with the requester-driven signals as inputs and `PREADY`/`PRDATA`/`PSLVERR` as outputs, plus `PAUSER`, `PWUSER`, `PWAKEUP` in and `PRUSER`, `PBUSER` out, one bit each (the fabric USER width). `PAUSER[0]`/`PWUSER[0]` become `awuser`/`aruser`/`wuser` on the fabric – observable at an `axil5` slave with `user` enabled. `PWAKEUP` is requester-driven (an input here, since the bridge is the completer) and is accepted and terminated: the bridge never waits for it. Behind the surface sits `apb5_to_axi4` (`apb4_to_axi4` for `protocol = "apb"`), then the same AXI4 timing wrapper an AXI4 master port gets; from there the port is an AXI4-Lite-shaped single-beat requester and takes the wide-slave aligner toward wider slaves. An unmapped address answers `PSLVERR` (the subtractive slave's DECERR folds into APB's one error bit).

15.3.5 AXI5-Lite Slave Surface

An `axil5` slave gets `axi4_to_axil5_{rd,wr}` at the boundary: the AXI4-Lite conversion core plus the AXI5-Lite ID and optional signals. The signal set is IHI 0022L B2.1.5 / Table B2.2: AXI5-Lite has **no** AxLOCK (no exclusive access), AxMPAM, AxMECID, AxNSAID or xLOOP – the bridge carried all five until GitHub #102 item 3 – and it **does** carry AWID/BID and ARID/RID. Because the master side is AXI4, the rest splits three ways, and which group a signal falls in is the whole story of what an AXI4 front end can and cannot express.

AXI5-Lite sideband groups at an `axil5` slave boundary

Group	Signals	Behaviour
ID	`awid`, `arid` (request); `bid`, `rid` (response)	The fabric ID (`{master index, id}`, the crossbar ID width), carried across the converter and returned unchanged.
FORWARDED	`awuser`, `wuser`, `aruser` (request); `buser`, `ruser` (response)	AXI4 has an equivalent, passed straight through and returned to the master. AXI4 `AxLOCK` has nowhere to go: an exclusive access completes as a normal one with OKAY.
TIED	`awtrace`, `wpoison`, `artrace`	AXI5-Lite optional signals with no AXI4 source. The port exists and is driven to `'0` — never left floating. There is deliberately no `ENABLE_` knob: there is nothing it could switch between.
TERMINATED	`btrace`, `rtrace`, `rpoison`	Completer-driven, with nothing on the AXI4 side to return them to. Accepted and dropped.

15.4 Functional Description

Droppable sideband passes natively end-to-end when both ends of a path are AXI5, feature-enabled, and width-matched; on any other path it terminates at the fabric boundary with a generation-time warning.

Connectivity-gated features are a config **error** unless *every* connected path is native (AXI5 both ends, feature enabled both ends, data widths matched — dwidth converters cannot carry per-beat sideband). Dropping POISON silently would launder corrupted data; dropping ATOP would turn an atomic into a plain write; dropping a tag operation would turn a tagged write into an untagged one and a Match into a plain access, with the requester none the wiser — so `mte` is gated the same way.

Chunking is permission, not demand. `ARCHUNKEN` tells the completer it *may* return the burst in chunks; a completer that cannot simply returns ordered data with `RCHUNKV` low, and every chunking requester must accept that. So `chunking` is droppable: toward an AXI4 slave, or through a width converter, the enable terminates with a generation-time warning and the read completes unchunked. Toward an AXI5 slave that chunks, the enable and the chunk fields ride the structs natively; the fabric never looks inside them — it routes each R beat by ID and frees its tracking on `RLAST` exactly as for an ordered burst, so chunks may arrive in any order the completer chooses.

Both need a 128-bit bus. Tags are one per 16 bytes and chunks are 128 bits; the validator rejects `mte` or `chunking` on a narrower port (`bridge_2x2_axi5_native` and `bridge_1x2_rd_axi5c` are the 128-bit fixtures; MAS 7.1a lists what their tests check).

Atomics depend on the port having a read path. `AWATOP = 01xxxx` (AtomicStore) and plain writes forward natively on any atomic-enabled port. Read-return classes (AtomicLoad `10xxxx`, AtomicSwap/Compare `11000x`) answer on the R channel with the AW's ID:

- On an **rw** master port they forward natively. The slave performs the operation and returns the location's original data on R; the bridge routes that beat back by ID (`axi5_atomic_rr_tracker` at the slave adapter, an extra AR->R tracking slot at the master adapter). Every connected atomic slave must be `rw` – the validator rejects a write-only one, since it could never return the data.
- On a **write-only** master port there is nowhere to deliver the R beat, so the boundary's `axi5_atomic_filter` answers those classes locally with **DECERR** – no slave-side AW handshake, no memory side effect. IHI 0022L A6.4.8 also requires the read data transfers to be sent even when the write response is an error; the filter generates them (GitHub #102 item 2), but a write-only port has no R channel to carry them, so the bridge accepts and **discards** them inside the filter (its `s_rready` is tied high, which also keeps its read-return queue from filling and stalling AW). The atomic therefore

completes with DECERR on **B only**. A manager that issues a load-class atomic on a write-only port is outside the spec – such a port has no read data channel to receive the result – and **must not** issue them; the DECERR exists so a stray one cannot hang the port. To use load-class atomics, declare the port `rw`.

Unique-ID indicators are regenerated, not routed back. IHI 0022L A5.2 makes `BIDUNQ` equal the request's `AWIDUNQ`, `RIDUNQ` equal `ARIDUNQ`, and an atomic's `RIDUNQ` equal its `AWIDUNQ`. The request indicator is droppable sideband: an AXI4 slave, a dwidth converter, the subtractive DECERR slave and the atomic filter's local response never see it. So the response indicators do not ride the fabric; each AXI5 master port with `unique` keeps one bit per ID, written at every AW (and AR, and read-return atomic AW) handshake and read by `BID`/`RID`. A5.2 forbids an `AxIDUNQ=1` request while any same-ID request is outstanding, and any request reusing an outstanding `AxIDUNQ=1` ID, so every outstanding transaction of one ID carries the same indicator and the per-ID bit is exact under any response order. On a write-only atomic port the table sits on the wrapper side of the filter, so a local DECERR carries the right bit.

AWAKEUP (IHI 0022L A14.1.1, feature `wakeup`). A master port accepts `AWAKEUP` and does not wait for it – the bridge is always clocked, and a Subordinate is permitted, not required, to wait. A wake-capable AXI5 slave port drives `AWAKEUP` as the OR of (a) every connected wake-capable master's request – the master's `AWAKEUP` OR'd with requests held in its boundary wrapper – so the slave can wake before a request reaches it, which is the A14.1.1 recommendation; the bridge cannot know which slave a request decodes to before it arrives, so every connected wake-capable slave sees it; and (b) the slave wrapper's own held requests, so `AWAKEUP` is high on every cycle the port presents `AWVALID`, `WVALID` or `ARVALID` and stays high until the handshake, as A14.1.1 requires – including traffic from a master without `AWAKEUP` (A14.1.1's "derive from AxVALID" fallback). Every term is a flop or a top-level `AWAKEUP` input, so the OR is glitch-free (A14.1). It falls when nothing is requested. The read and write halves of a port each produce a copy and the port ORs them: `AWAKEUP` is one signal for both request channels.

The tied group is the honest limit of an AXI4 front end. MPAM partition IDs, MECID encryption contexts and NSAID security IDs are properties of the *originating* master; a bridge whose masters speak AXI4 has nowhere to learn them, so an `axi5` slave reached from an `axi4` master sees zeros rather than invented values. If a design needs real MPAM or MECID at the slave, the master port has to be `axi5`, not `axi4`. (AXI5-Lite has no MPAM, MECID or NSAID at all.)

Sideband is held for the whole burst, not just the address beat. AXI4 presents `AWID`/`AWUSER` once, with AW; AXI5-Lite needs them stable across every beat that follows. The shims capture on the address handshake and hold:

```
wire w_aw_accept = s_axi_awvalid && s_axi_awready;
wire [AXI_USER_WIDTH-1:0] w_awuser_sel = w_aw_accept ? s_axi_awuser
                                                     : r_held_awuser;
```

A combinational passthrough here is a real defect rather than a style question: with two bursts in flight, burst A's later beats would carry burst B's `AWUSER`.

15.4.1 Interop Matrix

Interop Matrix

Master Slave	axi4	axi5	apb / apb5 / axil	axil5
axi4	native	base subset	via shim	via shim, sideband TIED to `'0`
axi5	sideband drops (warning)	**native sideband** when width-matched	sideband drops (warning)	forwarded where AXI4-expressible
axil / axil5	single-beat AXI4	single-beat AXI4	via shim	ID and `user` forwarded end to end (axil5 master); rest tied
apb / apb5	single-beat AXI4	single-beat AXI4	via shim (APB in, APB out)	`PAUSER[0]` visible as `awuser` (apb5 master)

Connectivity-gated features tighten the axi5→axi5 cell: they *require* it.

16 Unmapped-Address Handling

16.1 Overview

16.1.1 Why the bridge answers an address no slave owns

A positively-decoded fabric answers only addresses that fall inside some slave's range. An address in none of them selects nothing: the one-hot select is all-zero, no slave sees `AWVALID`/`ARVALID`, `READY` never rises, and the master waits forever.

A hang is the worst failure available here, because it destroys the evidence. There is no response to inspect, no error bit to read, and the offending address is whatever the master still has latched. A stray pointer in software becomes a wedged interconnect with nothing to point at the cause.

The bridge therefore ends its decode chain in an `else` – subtractive decode – and that `else` selects an internal slave that always answers.

16.2 Ports

16.2.1 Status, interrupt and clearing

The catch-all latches its status. Every bridge exposes:

Unmapped-address status and interrupt ports

Port	Dir	Meaning
`unmapped_irq`	out	**sticky**: set by the first unmapped access, held until cleared
`unmapped_addr[31:0]`	out	address of that **first** access
`unmapped_count[7:0]`	out	number of unmapped accesses, saturating at 255
`unmapped_clear`	in	pulse to clear the flag and the count

Three choices worth knowing about:

- **Sticky, not a pulse.** A one-cycle pulse is gone before software can look, and this is precisely the access nobody expected.
- **The first address wins.** A later fault does not overwrite it; the first one is usually what explains the rest.
- **The address survives the clear**, so a late reader still learns where the fault was after the flag is acknowledged. The count saturates rather than wrapping – “255+” is honest, a wrapped 3 is not.

16.3 Functional Description

16.3.1 Behaviour

Subtractive-slave response behaviour

Write	every W beat is accepted; one B per AW
Read	AxLEN+1 beats, RLAST on the last
Response	**DECERR** on both, always
Read data	0xDEADBEEF, replicated to the bus width

DECERR rather than SLVERR is the AXI4 code for "no slave at this address": nothing failed to service the request, there was nothing there to service it. There is no OKAY mode and no parameter to select one – a mode switch here is where lint, simulation and synthesis end up disagreeing about what the fabric does.

The read data is a recognisable pattern rather than zeros because zeros are indistinguishable from real memory. 0xDEADBEEF in a dump is a fault; 0 is a plausible value.

Write data is discarded. There is nowhere for it to go, and inventing a destination would be worse than saying so.

16.3.2 Register access (cfg regblock builds)

Bridges built with the cfg register block also expose the status over the cfg **AXI4-Lite** window. The port is s_cfg_axil_* – AXI4-Lite, not APB. An APB-attached CPU reaches it through an axi4_to_apb4_shim upstream of the bridge, which is an integration choice and not part of this module:

Unmapped-address status registers

Register	Field	Access	Meaning
SUBTRACTIVE_STATUS	HIT[0]	RO	mirrors unmapped_irq
	CLEAR[1]	W	write 1 to clear the flag and count
	COUNT[15:8]	RO	mirrors unmapped_count
SUBTRACTIVE_ADDR	ADDR[31:0]	RO	mirrors unmapped_addr

The sticky state lives in the RTL, not in the register block: one owner for one piece of state, so the two cannot disagree about whether a fault happened. These registers are a window onto it plus a clear. Clearing works from either the `unmapped_clear` pin or a `CLEAR` write; neither disables the other.

These registers are appended **after** every pre-existing register, so adding them did not move any existing offset. That is deliberate: a register map is an ABI. An earlier attempt inserted per-slave configuration for the catch-all midway through the map, which renumbered everything after it, silently moved the monitor-enable bits, and left the monitors switched off while every functional test still passed.

16.3.3 When it can never fire

The catch-all is the `else` of the decode chain, so it is reachable only if the slave ranges leave a **gap**. A map that tiles the whole address space has no unmapped address to catch, and the `else` is unreachable logic that synthesis removes – it costs nothing and reports nothing.

Most shipped variants are in exactly that position. Of the 22 generated bridges, **18 tile the space completely** and 4 leave gaps. `bridge_2x2_rw` is typical of the first group: `ddr` takes `<= 0x7FFF_FFFF` and `sram` takes `>= 0x8000_0000`, between them the entire 32-bit range.

This is worth knowing before wiring `unmapped_irq` to anything: on a fully-tiled bridge it is tied low by construction, and a test that expects it to assert will wait forever. Check your own address map, not this table – it changes with the TOML.

The insurance is still worth having. A map is tiled *today*; the next slave added, or the next range narrowed, opens a gap, and the difference between “reports it” and “hangs the master” is decided at that moment rather than at integration time.

16.3.4 What this does not do

The catch-all does not make an unmapped access *correct*. It makes it **visible and survivable**: the master gets an error instead of stalling, and software gets an address instead of a puzzle. An address that reaches it is still a configuration or software fault.

16.4 Navigation

Previous: AXI5 and APB5 Interfaces

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

17 Wishbone B4 Interface

17.1 Overview

Wishbone B4 is the open bus of the FPGA world – the fabric OpenCores cores speak and the one most soft CPUs ship with. Since bridge TASK-008 (was BRIDGE-019) a bridge port can be Wishbone on either side: a **slave port** with `protocol = "wb4"` makes the bridge the Wishbone requester toward an external completer; a **master port** with `protocol = "wb4"` makes the bridge the completer for an external Wishbone requester. The fabric between stays AXI4.

The mode is **B4 pipelined** – a new STB every clock while STALL is low, in-order termination – which is what `rtl/amba/wb4` and the converters implement. B4 standard (“classic”) mode exists in the converters as a parameter but is not selectable from the TOML; the two modes do not mix on one bus, so a classic peer needs a classic bridge build.

17.2 Ports

17.2.1 Signal Definition

Wishbone B4 Signal Definitions

Signal	Width	Slave port (bridge requests)	Master port (bridge completes)	Description
CYC	1	Output	Input	Bus cycle in progress
STB	1	Output	Input	Transfer request
WE	1	Output	Input	1 = write, 0 = read
ADR	ADDR_WIDTH	Output	Input	Byte address (the fabric address, unwindowed)
DAT_W	DATA_WIDTH	Output	Input	Write data (the spec's DAT_O from the requester)
SEL	DATA_WIDTH/8	Output	Input	Byte lane select = AXI WSTRB
CTI	3	Output	Input	Burst hint (carried, not acted on; CLASSIC when the bridge drives it)
BTE	2	Output	Input	Burst type extension (LINEAR when the bridge drives it)
STALL	1	Input	Output	Pipelined back-pressure
ACK	1	Input	Output	Normal termination
ERR	1	Input	Output	Error termination
RTY	1	Input	Output	Retry termination (never driven by the bridge)
DAT_R	DATA_WIDTH	Input	Output	Read data (the spec's DAT_I at the requester)

17.2.2 Signal Naming

Wishbone ports use the TOML prefix followed by the uppercase B4 name, the convention `rtl/amba/wb4` and the RDS-DV BFMs share:

```
// Slave port, prefix "wbp_wb_" -- the bridge drives the bus
output logic        wbp_wb_CYC,
output logic        wbp_wb_STB,
output logic        wbp_wb_WE,
output logic [31:0] wbp_wb_ADR,
output logic [31:0] wbp_wb_DAT_W,
output logic [3:0]  wbp_wb_SEL,
output logic [2:0]  wbp_wb_CTI,
output logic [1:0]  wbp_wb_BTE,
input  logic        wbp_wb_STALL,
input  logic        wbp_wb_ACK,
input  logic        wbp_wb_ERR,
input  logic        wbp_wb_RTY,
input  logic [31:0] wbp_wb_DAT_R
```

A master port has the same thirteen with the directions reversed.

17.3 Functional Description

17.3.1 Slave port: AXI4 to Wishbone

`axi4_to_wb4` (converters MAS, chapter 3) sits in the slave adapter: the AXI4-Lite decomposers turn each AXI4 burst into one single-beat transfer per beat, and `axil4_to_wb4` turns those into Wishbone transfers, in order. `WSTRB` becomes `SEL`. Termination maps back as:

Wishbone termination to AXI4 response

Wishbone	AXI4 response
ACK	OKAY
ERR	SLVERR
RTY	SLVERR (`RTY_RESP`; AXI has no retry)

The monitored (`mon`) variant places the `axi4_master_*_monlite` wrappers between the crossbar and the converter, exactly as for APB slave ports.

17.3.2 Master port: Wishbone to AXI4

`wb4_to_axi4` sits in the master adapter in front of the ordinary AXI4 timing wrapper: `wb4_to_axil4` merges AXI's two response channels back into Wishbone's single in-order termination stream, and the wrapper promotes the AXI4-Lite face to AXI4 (`AxLEN = 0`, full-width `AxSIZE`, INCR, `WLAST = 1`, a constant ID – the fabric prepends the master index, bridge TASK-005 (was BRIDGE-016)). From the wrapper on, the port is an AXI4-Lite-shaped single-beat requester: decode, the width converters, the wide-slave aligner toward wider slaves and the response mux are the ones every other master uses. `SLVERR` and `DECERR` both terminate `ERR` – an unmapped address (the subtractive slave) is indistinguishable from a failing completer on this bus. `RTY` is never generated.

17.3.3 Rules the validator enforces

- `channels = "rw"` (one bus carries both directions), `id_width = 0`.
- `data_width` is 8, 16, 32 or 64.
- A master port has `addr_width = 32`: `ADR` is the full fabric address, where an APB or Wishbone *slave* port's address is whatever the fabric forwards for its window.

17.4 Design Notes

- **In-order by construction.** Wishbone terminates in issue order and the bridge's slave-side tracking is in-order for shim-converted slaves, so a Wishbone completer never needs the CAM. Two masters sharing a Wishbone completer are serialised by the crossbar's arbitration and answered in that order.
- **One transfer per beat.** A 16-beat AXI4 burst is sixteen Wishbone transfers; the pipelined bus takes one per clock when the completer does not stall, so the cost is latency, not bandwidth.
- **Burst hints are not generated.** `CTI`/`BTE` are CLASSIC/LINEAR on a slave port; a Wishbone completer that needs registered-feedback bursts to perform will not see them from the bridge.
- **Best effort, by decision (2026-09-13).** The Wishbone ports are spec-legal and regress green, and that is the accepted state: forming pipelined Wishbone bursts from AXI bursts and putting a Wishbone slave in its own clock domain (`cdc` is AXI4-only) are not owed. They cost only Wishbone throughput nobody has asked for; a consumer that needs either files a task against that need.

Verified by `dv/tests/test_bridge_2x2_wb4_paths.py` on `bridge_2x2_wb4` (error folding both ways, SEL lanes at both a Wishbone and a 64-bit AXI4 completer, burst decomposition, both requesters in flight at the Wishbone completer) plus the generated tests of that fixture.

18 Throughput Characteristics

18.1 Overview

What the fabric can move when everything goes right, and what eats into that number when it doesn’t. The short version: bursts and matched widths win; conversions and contention cost you.

18.2 Functional Description

18.2.1 Peak Throughput

One master driving one slave — no contention, no conversion:

Peak Throughput by Data Width

Data Width	Frequency	Peak Throughput
32-bit	100 MHz	400 MB/s
64-bit	100 MHz	800 MB/s
128-bit	100 MHz	1.6 GB/s
256-bit	100 MHz	3.2 GB/s
512-bit	100 MHz	6.4 GB/s

18.2.2 Formula

```
Peak Throughput = DATA_WIDTH (bits) × Frequency (Hz) / 8 bytes/bit
```

18.2.3 Measured (2026-09-11)

`bridge_2x2_rw` (two 32-bit AXI4 masters, two 32-bit AXI4 slaves, no width or protocol conversion), 16-beat INCR bursts, every BFM channel back-to-back, each figure computed over its own window from the first to the last beat at the port named. `test_bridge_2x2_rw_perf` asserts a floor under every row, so a change that costs bandwidth fails there instead of ageing this table.

Measured throughput, direct 32-bit AXI4 paths

Traffic	Where measured	Beats / cycle	Notes
One master streaming reads	R at the master port	**1.00**	128 bursts, 2048 beats in 2048 cycles
One master streaming writes	W at the slave port	**0.89-0.90**	0 cycles of WREADY low at the master port: the ~2-cycle gap per burst is the requester re-arming W between bursts, not the fabric
Two masters	W at the shared	**1.00**	the port stays full;

Traffic	Where measured	Beats / cycle	Notes
streaming writes to one slave	slave port		the other master's beats fill the gaps above
– per-master share of the contended window	W at each master port	**0.499 / 0.501**	128 bursts each (0.475 / 0.525 at 8 bursts: round-robin, window-length effects only)
Two masters to two different slaves	W at both slave ports	**1.78-1.80**	two independent paths; neither slows the other

So the formula above holds for this fabric: a saturated port moves one beat per cycle, two masters on one port split it evenly, and paths to different slaves do not share anything. The write-stream row is the requester's figure, not the bridge's – the contention row is the bridge's.

The current performance tests measure throughput floors under uniform-style patterns; a full saturation study (accepted traffic vs. applied load, latency vs. offered load) is tracked as bridge vault TASK-014.

18.2.4 Factors Affecting Throughput

Factors Affecting Throughput

Factor	Impact	Mitigation
Arbitration contention	Reduces per-master throughput	Increase slave ports
Width conversion	UNVERIFIED – see note	Match widths where possible
Protocol conversion	2+ cycles for APB	Use AXI4 for high-bandwidth
Response routing	Minimal (pipelined)	N/A

> **The width-conversion figure is not measured.** "1-cycle penalty per direction" has no source: the converters (`axi_data_upsize`, `axi_data_dnsize`, `axi4_dwidth_converter_{rd,wr}`) decompose or combine beats, so the cost depends on the width RATIO and the burst length, and a single constant cannot be right for 256b->32b and 64b->32b alike.

It is left marked rather than replaced with a guess. Measuring it needs a propagation measurement on a converted path in `bridge_4x4_rw` (gpu 256b -> periph 32b) against a matched one (cpu 64b -> ddr0 64b), in the style of `test_bridge_2x2_rw_latency`. An attempt at that test could not reliably observe the master-side reference and was withdrawn rather than committed half-working.

18.2.5 Multi-Master Scaling

With fair round-robin arbitration (measured above: 0.499 / 0.501 at the shared port; the sum stays at one beat per cycle):

```
Per-Master Throughput = Peak Throughput / Active Masters (to same
slave)
```

18.2.5.1 Example: 4 Masters, 2 Slaves

```
All 4 masters accessing Slave 0:
  Per-master = Peak / 4

2 masters to Slave 0, 2 masters to Slave 1:
  Per-master = Peak / 2 (full parallelism)
```

18.2.6 Burst Efficiency

Burst Efficiency Comparison

Transaction Type	Overhead	Efficiency
Single beat	1 cycle address + 1 cycle data	50%
4-beat burst	1 cycle address + 4 cycle data	80%
16-beat burst	1 cycle address + 16 cycle data	94%
256-beat burst	1 cycle address + 256 cycle data	99.6%

18.2.7 Width Conversion Impact

18.2.7.1 Upsize (Narrow to Wide)

```
64-bit to 512-bit (8:1 ratio):
  Input: 8 beats × 64 bits = 512 bits
  Output: 1 beat × 512 bits = 512 bits
```

```
Throughput: Preserved (same data, fewer beats)
Latency: +1 cycle (packing delay)
```

18.2.7.2 Downsize (Wide to Narrow)

```
512-bit to 64-bit (8:1 ratio):
  Input: 1 beat × 512 bits = 512 bits
  Output: 8 beats × 64 bits = 512 bits

Throughput: Preserved (same data, more beats)
Latency: +7 cycles (sequential output)
```

18.2.8 Protocol Conversion Impact

AXI4 vs APB Protocol Impact

Metric	AXI4 Direct	AXI4 to APB
Min cycles/transfer	1	2
Burst support	Yes	No (split)
Pipeline depth	1+	1
Peak throughput	DATA_WIDTH/cycle	DATA_WIDTH/2 cycles

18.3 Design Notes

Bursts buy you efficiency:

- Use bursts whenever possible
- AXI4 supports up to 256 beats per burst
- APB does not support bursts (split internally)

And protocol choice matters more than most people expect:

- Keep high-bandwidth traffic on AXI4 paths
- Use APB only for low-bandwidth peripherals
- Group APB slaves to avoid impacting AXI4 traffic

18.4 References

- Duato/Yalamanchili/Lionel, *Interconnection Networks*, Morgan Kaufmann 2003 — the standard network-evaluation methodology plots accepted traffic vs. applied load and latency vs. applied load, normalized to network capacity (ch9). (Reference archive)
- Dally & Towles, *Principles and Practices of Interconnection Networks*, Morgan Kaufmann 2004 — uniform-random traffic is benign; permutation, hot-spot, and adversarial patterns are the stresses that reveal allocation limits (ch3 §3.2-3.3). (Reference archive)

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

19 Latency Analysis

19.1 Overview

Where the cycles go on the way through the bridge: address path, data path, response path, then the end-to-end best cases and the conversion costs. The headline numbers for a direct AXI4 path are measured, not estimated — the note after Table 5.7 explains how, and why an earlier version of that table was wrong.

The current latency measurements are best-case and lightly-loaded figures; a full latency-vs-offered-load sweep to find the saturation knee is tracked as bridge vault TASK-014.

19.2 Timing

19.2.1 Address Path Latency

Address Path Latency

Stage	Cycles	Notes
Master Adapter (skid)	1	Pipeline registration
Address Decode	0	Combinational
Arbitration	0-1	0 if no contention, 1 if arbitrate
Slave Router	1	Pipeline registration
Total Address	**2-3**	Best/worst case

19.2.2 Data Path Latency

Data Path Latency

Stage	Cycles	Notes
W follows AW	0	Same grant
Width conversion	0-1	0 if match, 1 if convert
Protocol conversion	0-2+	0 for AXI4, 2+ for APB
Total Data	**0-3+**	Best/worst case

19.2.3 Response Path Latency

Response Path Latency

Stage	Cycles	Notes
Slave response	Variable	Slave-dependent
ID extraction	0	Combinational – routing is by FIFO position, no lookup
Response routing + master delivery	2	Registered; NOT a direct connection
Total Response	**2 + slave**	Plus slave latency

Measured. `bridge_2x2_rw`, cpu master to ddr slave, idle bridge:

Path	Cycles
AW accepted at the master -> AWVALID at the slave port	**2**
B accepted at the slave port -> BVALID at the master	**2**

Two skid stages each way: the master adapter's `axi4_slave_wr` AW skid, then the slave adapter's `axi4_master_wr` AW skid. The response path has the same structure, which is why the two figures match.

The response row previously read "Master delivery 0 (Direct connection)", total 1. It is registered, and the figure is 2.

What this measures, and what an earlier revision of this note got wrong. These are PROPAGATION times – how long a beat takes to appear on the far side. An earlier revision quoted 4 cycles for the request path, measured accept-to-accept. That metric also counts however long the far side held READY low, so it describes the attached slave's timing as much as the bridge's, and it was not even stable: successive runs gave 4/2 and then 2/6. Propagation is 2/2 on every run.

`test_bridge_2x2_rw_latency` asserts both values EXACTLY, so adding or removing a pipeline stage fails the test rather than silently ageing this table.

Other configurations differ – width converters and the APB/AXIL shims add stages – so this is the figure for a direct AXI4 path, not a universal constant.

With the registered crossbar (`[bridge] xbar_pipeline = true`, bridge TASK-006 (was BRIDGE-017)) both figures are **3**: one skid stage each way inside the xbar. `test_bridge_2x2_rw_pipe_latency` asserts 3/3 on `bridge_2x2_rw_pipe` exactly as the baseline test asserts 2/2, and `test_bridge_2x2_rw_perf` runs its four phases on both fixtures with the same floors – the stage costs a cycle of latency and no bandwidth.

19.2.4 Loaded Write Latency (measured)

`test_bridge_2x2_rw_perf` streams 16-beat write bursts from one master and pairs the k-th AW accepted at the master port with the k-th BVALID there:

Write latency under load, `bridge_2x2_rw`, 32-bit direct path

Queue	AW accepted -> BVALID (cycles)
First burst, fabric empty	**23** = 16 W beats + the 2 + 2 skid stages + the slave's turnaround
k bursts queued ahead	about 16 k + 7; maximum 323 at a 20-deep queue

The queue exists because the bridge accepts AWs ahead of their W data – the master adapter's AW skid, the slave adapter's 16-entry response-tracking FIFO and its AW skid together hold about 20 bursts – so a requester that issues many AWs before its data sees its B responses spaced by the data time, not by the fabric. Read latency does not queue the same way: the R stream is one beat per cycle end to end (Table 5.1a).

19.2.5 End-to-End Latency

19.2.5.1 Write Transaction (Best Case)

```
Cycle 0: AW arrives at Bridge
Cycle 1: AW exits to Slave (skid buffer)
Cycle 2: Slave receives AW
Cycle 1: W arrives (same cycle as AW skid)
Cycle 2: W exits to Slave
Cycle 3: Slave receives W, generates B
Cycle 4: B routed back to Master

Total: 4 cycles (minimum)
```

19.2.5.2 Read Transaction (Best Case)

```
Cycle 0: AR arrives at Bridge
Cycle 1: AR exits to Slave (skid buffer)
Cycle 2: Slave receives AR, generates R
Cycle 3: R routed back to Master

Total: 3 cycles (minimum)
```

19.2.5.3 Contention Latency

When multiple masters contend for same slave:

```
Arbitration adds 1 cycle per contending master

Example: 4 masters requesting same slave
  Worst case: Wait for 3 other masters = 3 extra cycles
  Average case: Wait for 1.5 masters = 1-2 extra cycles
```

19.2.6 Width Conversion Latency

19.2.6.1 Upsize Latency

Upsize Latency by Ratio

Ratio	Extra Cycles	Notes
1:1	0	No conversion
2:1	0	Pack on fly
4:1	0	Pack on fly
8:1	0-1	May need accumulation

19.2.6.2 Downsize Latency

Downsize Latency by Ratio

Ratio	Extra Cycles	Notes
1:1	0	No conversion
1:2	+1	2 output beats
1:4	+3	4 output beats
1:8	+7	8 output beats

19.2.7 Protocol Conversion Latency

AXI4 to APB Conversion Latency

Phase	Cycles
AXI4 AW decode	1
APB setup	1
APB access	1+ (PREADY)
B generation	1
Total	**4+ cycles**

19.2.7.1 APB Wait States

APB slaves may insert wait states:

```
PREADY deasserted: +1 cycle per wait
Typical UART/GPIO: 0-2 wait states
Slow peripherals: May have many wait states
```

19.3 Design Notes

19.3.1 Design Recommendations

1. **Match data widths** - Avoid conversion latency
2. **Use AXI4 for fast paths** - Avoid APB latency
3. **Minimize contention** - Spread traffic across slaves
4. **Use bursts** - Amortize address latency
5. **Pipeline responses** - Don't block on slow slaves

19.4 References

- Duato/Yalamanchili/Lionel, *Interconnection Networks*, Morgan Kaufmann 2003 — accepted traffic vs. applied load and latency vs. applied load are the standard evaluation plots for finding the saturation point (ch9). (Reference archive)
- Dally & Towles, *Principles and Practices of Interconnection Networks*, Morgan Kaufmann 2004 — uniform-random traffic is benign; permutation and hot-spot patterns expose the allocation limits that matter for worst-case latency (ch3 §3.2-3.3). (Reference archive)

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

20 Resources

20.1 Overview

What a generated bridge costs on an FPGA, measured. The numbers below come from the out-of-context synthesis and implementation flow in `projects/components/fabric-gen-ip/bridge/fpga/` (HAS 6.4: how it constrains the bridge, how to repeat a run, how to read a line of `reports/summary.csv`), on the two parts this repository's boards carry. Earlier revisions of this page carried hand estimates per component and per configuration; they did not reconcile with each other and had never been checked against a synthesis report, so they are gone. Anything not in the tables below is a configuration nobody has synthesized yet, and the flow takes minutes to answer for it.

20.2 Measured Utilization and Timing

Measured utilization and timing, Artix-7 100T -1 at 10 ns (HAS 6.4 flow)

Bridge	LUTs	FFs	BRAM	WNS reg-to-reg (ns)	Fmax est. (MHz)	Worst logic levels
bridge_2x2_axi5	4,441	3,325	0	+0.40	104.2	8
bridge_2x2_axi5_native	8,007	6,213	0	+0.04	100.4	12
bridge_2x2_rw	4,615	3,253	0	+0.12	101.2	9
bridge_2x2_rw_cdc	4,851	3,413	0	+0.27	102.7	9
bridge_2x2_rw_pipe	5,646	4,227	0	+1.27	114.6	6
bridge_2x2_rw_qos	5,136	3,317	0	-2.91	77.4	14
bridge_2x2_rw_qos_pipe	5,494	4,291	0	+0.17	101.7	9
bridge_4x4_rw	30,278	29,508	0	-1.57	86.4	14
bridge_5x3_channels	19,243	16,932	0	-2.66	79.0	16
bridge_mix_a	6,456	5,697	0	-0.29	97.1	16

Measured utilization and timing, Kintex-7 325T -2 at 6.667 ns

Bridge	LUTs	FFs	BRAM	WNS reg-to-reg (ns)	Fmax est. (MHz)	Worst logic levels
bridge	4,430	3,325	0	+0.69	167.4	9

Bridge	LUTs	FFs	BRAM	WNS reg-to-reg (ns)	Fmax est. (MHz)	Worst logic levels
_2x2_axi5						
bridge_2x2_axi5_native	7,969	6,213	0	+0.26	156.1	12
bridge_2x2_rw	4,612	3,253	0	+0.97	175.6	9
bridge_2x2_rw_cdc	4,842	3,413	0	+0.55	163.4	9
bridge_2x2_rw_pipe	5,647	4,227	0	+1.73	202.6	6
bridge_2x2_rw_qos	4,963	3,317	0	-0.18	146.0	13
bridge_2x2_rw_qos_pipe	5,475	4,291	0	+0.70	167.6	9
bridge_4x4_rw	29,719	29,508	0	+0.28	156.5	13
bridge_5x3_channels	19,035	16,932	0	-0.13	147.1	17
bridge_mix_a	6,375	5,697	0	+0.53	162.9	8

Fmax is an estimate from the register-to-register slack at the constrained period; the I/O budget and the reading of each row are in HAS 6.4. The rows that miss on the Artix-7 all fail on one path – master adapter, crossbar arbitration and mux, slave CAM allocate – and `xbar_pipeline` is the register that splits it: `bridge_2x2_rw_qos` at -2.91 ns becomes `bridge_2x2_rw_qos_pipe` at +0.17 ns.

20.2.1 Scaling

Resource Scaling

Component	Scales with
Crossbar core	O(M x N) muxing, plus one arbiter per slave channel
Master adapters	O(M), each holding a full beat per channel in its skid buffers
Slave adapters	O(N), plus a converter and its monitor sandwich for every non-AXI4 or width-stepped port
bridge_id FIFOs	O(N x depth), small LUT arrays, no BRAM
Registered crossbar (`xbar_pipeline`)	one 2-deep skid per slave-side channel
CDC slave port (`cdc`)	one async FIFO per channel of that port

Block RAM and DSP usage are zero in every configuration measured: the per-slave bridge_id FIFOs and the CAM are LUT arrays, and there is no arithmetic.

The current resource figures come from a small set of configurations driven with uniform-style traffic; a systematic resource-vs-load characterization across adversarial patterns is tracked as bridge vault TASK-014.

20.3 Design Notes

20.3.1 Resource Reduction

1. **Use channel-specific masters** - a read-only or write-only master builds half the adapter and half the crossbar columns
2. **Match data widths** - a width step costs a converter each way
3. **Use APB and AXI-Lite slaves sparingly** - each carries a shim and a second monitor
4. **Limit outstanding transactions** - shallower per-slave bridge_id FIFOs

20.3.2 Performance/Resource Trade-off

Performance/Resource Trade-offs

Feature	Resource Impact	Performance Impact
Deeper skid buffers	+registers	+timing margin
Deeper bridge_id FIFOs	+LUTs	+outstanding depth (no OOO support)
Pipeline stages	+registers	+frequency
Wider data paths	+routing	+throughput

20.4 References

- Duato/Yalamanchili/Lionel, *Interconnection Networks*, Morgan Kaufmann 2003 — accepted traffic vs. applied load and latency vs. applied load are the standard network-evaluation plots; resource counts must be paired with those load sweeps to be meaningful (ch9). (Reference archive)
- Dally & Towles, *Principles and Practices of Interconnection Networks*, Morgan Kaufmann 2004 — uniform-random traffic is benign; permutation and hot-spot patterns are the standard stresses that reveal allocation and buffer-sizing limits which in turn drive resource growth (ch3 §3.2-3.3). (Reference archive)

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

21 System Requirements

21.1 Overview

Everything your system owes the bridge before it will do its job: one clean clock, an asynchronous active-low reset, a single power domain, interfaces that follow the AXI4 rules, and an address map with no overlaps. Gaps in the map are allowed — the bridge answers those addresses itself — and the checklist at the end tells you how to prove all of it.

21.1.1 Clock Infrastructure

Clock Requirements

Requirement	Specification
Clock source	One fabric clock (`aclk`) for the fabric and every master port; a `cdc = true` AXI4 slave port brings its own `<slave>_aclk` (HAS 4.5a)
Clock quality	Low jitter, stable frequency
Clock distribution	Must reach all Bridge ports

21.1.2 Reset Infrastructure

Reset Requirements

Requirement	Specification
Reset type	Asynchronous, active-low (aresetn)
Reset synchronization	External synchronizer recommended
Reset distribution	Must reach all Bridge ports

21.1.3 Power

Power Requirements

Requirement	Specification
Power domains	Single domain (no power gating)
Voltage	FPGA/ASIC process-dependent

21.2 Parameters

21.2.1 Address Map

Address Map Requirements

Requirement	Specification
Slave ranges	Must not overlap
Range alignment	Power-of-2 recommended
Coverage	Gaps are permitted: an address in no slave range is claimed by the internal subtractive slave and answered with DECERR (see 4.5). It is still a fault – it is reported, not tolerated.

21.2.2 Connectivity

Connectivity Requirements

Requirement	Specification
Matrix completeness	All masters must access at least one slave
Reachability	Consider traffic patterns for performance

21.3 Ports

21.3.1 Master Requirements

Masters connecting to Bridge must:

1. **Comply with AXI4 protocol** - Valid/ready handshake
2. **Use correct ID width** - Match configured ID_WIDTH
3. **Use correct data width** - Match configured DATA_WIDTH
4. **Use correct address width** - Match configured ADDR_WIDTH
5. **Stay within address range** - Target valid slave addresses

21.3.2 Slave Requirements

Slaves connecting to Bridge must:

1. **Comply with AXI4/APB protocol** - Based on configuration
2. **Accept the widened ID** - a slave sees `{master index, master id}`: the widest master's id_width plus `$clog2(NUM_MASTERS)` bits (none for a single master). Declare the slave's `id_width` at least that wide; the validator refuses less.
3. **Match configured data width** - Or use width conversion
4. **Respond to all transactions** - No hanging

21.4 Timing

21.4.1 Setup/Hold

Timing Requirements

Constraint	Requirement
Input setup	Meet FPGA/ASIC requirements
Input hold	Meet FPGA/ASIC requirements
Clock-to-output	Per process characterization

21.4.2 Recommended Margins

Recommended Timing Margins

Margin	Value	Purpose
Setup margin	10-20%	Variation tolerance
Hold margin	Positive	No hold violations
Clock uncertainty	Include in constraints	PLL/MMCM jitter

21.5 Testing

21.5.1 Pre-Integration Checks

1. **RTL lint clean** - No Verilator warnings
2. **Parameter validation** - All parameters in valid range
3. **Address map review** - no overlaps. Complete coverage is NOT required: a gap is claimed by the subtractive slave and answered with DECERR (4.5). Decide deliberately which you want – a gap that is meant to be a gap is fine, a gap you did not know about is a bug the fabric will now report.
4. **ID width calculation** - Sufficient for NUM_MASTERS

21.5.2 Integration Checks

1. **Signal connectivity** - All ports connected
2. **Width matching** - DATA_WIDTH, ADDR_WIDTH, ID_WIDTH
3. **Protocol matching** - AXI4 vs APB correctly configured
4. **Clock/reset connectivity** - All ports share same domain

21.5.3 Post-Integration Checks

1. **Basic transaction test** - Read/write to each slave
2. **Arbitration test** - Multi-master contention
3. **Error handling test** - unmapped-address response: DECERR + 0xDEADBEEF, `unmapped_irq` sticky, `SUBTRACTIVE_STATUS.CLEAR` clears it (see 4.5)
4. **Performance validation** - Meet throughput targets

21.6 Assumptions and Known Limitations

21.6.1 Response-Ordering Assumption

Bridge routes B and R by a sideband master index stored per slave. For an AXI4/AXI5 slave of a multi-master bridge (or any slave with `enable_ooo`) the store is `bridge_cam`, looked up by the returned BID/RID, so the slave may complete different IDs in any order (ARM IHI 0022E §A4). Every other slave – single-master bridges, AXI4-Lite/APB/Wishbone shim slaves and the internal subtractive slave – uses an in-order FIFO and must return responses in request order, which those slaves do by construction.

21.6.2 Buffer and CAM Depths

All adapter skid depths, response-tracking FIFO depths, and CAM dimensions are generation-time constants. There is no runtime sizing API. A systematic sizing rule that relates buffer depth to credit-loop latency, outstanding-transaction count, and width-converter buffering is tracked as bridge vault TASK-016.

21.7 References

- ARM, *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022E — AXI4 ordering rules allow out-of-order completion across IDs (§A4). (Reference archive)
- Maaref, *Architecting and Building High-Speed SoCs*, Packt 2022 — AMBA-suite context and the AXI ID ordering rules that Bridge's FIFO mode simplifies (ch3 §4.2). (Reference archive)
- Eschemann, *The FPGA Programming Handbook*, Packt 2023 — educational secondary source describing out-of-order reads as a normal AXI4 feature (ch8 §8.3). (Reference archive)

22 Parameter Configuration

22.1 Overview

Every bridge the generator emits starts here: port counts, bus widths, per-port configuration, and the address map. This chapter collects the full parameter set — the core knobs, the widths the generator derives from them, the per-port fields, the monitor options, and the checks the generator runs before it accepts your configuration.

22.2 Parameters

22.2.1 Core Parameters

Bridge Core Parameters

Parameter	Type	Range	Default	Description
NUM_MASTERS	int	1-32	2	Number of master ports
NUM_SLAVES	int	1-256	2	Number of slave ports
ADDR_WIDTH	int	12-64	32	Address bus width
DATA_WIDTH	int	32-512	64	Data bus width
ID_WIDTH	int	1-16	4	Master ID width
USER_WIDTH	int	1-16	1	User signal width

22.2.2 Registered Crossbar (`xbar_pipeline`)

Crossbar options (bridge TASK-006, was BRIDGE-017)

TOML key (`[bridge]`)	Type	Default	Effect
`xbar_pipeline`	bool	`false`	`true` puts a 2-deep skid stage on every slave-side channel (AW, W, AR, B, R) inside the crossbar. Propagation 3/3 cycles instead of 2/2 (Table 5.7), throughput unchanged. For high-fanout or wide fabrics that

TOML key ([bridge])	Type	Default	Effect
			miss timing on the arbiter-plus-mux or the response OR-merge cone. Measured (HAS 6.4): the register that splits the arbitration-mux-CAM path; on the Artix-7 at 100 MHz any configuration beyond a plain 2x2 – QoS, wide ports, more masters – needs it.
`arbitration`	`"rr"` / `"qos"`	`"rr"`	Per-slave arbitration policy. `"qos"` grants the highest `AxQOS` plus aging (MAS 2.3, Per-Slave Request Arbitration); it decides among requests pending at the arbiter, so it shows at a slave that backpressures AW, not at one that accepts every AW on arrival; equals share round-robin.
`qos_aging_shift`	int 0..7	4	With `"qos"`: a

TOML key ([bridge])	Type	Default	Effect
			waiting request gains one priority level every `2**shift` cycles, so the worst wait for a QoS-0 request is `15 * 2**shift` cycles.

CDC slave port option (bridge TASK-006, was BRIDGE-017)

TOML key ([[bridge.slaves]])	Type	Default	Effect
`cdc`	bool	`false`	`true` gives this AXI4 slave port its own clock: `<name>_aclk` / `<name>_aresetn` on the top, `axi4_cdc_{wr,rd}` between the adapter's timing wrapper and the port (HAS 4.5a). Slave ports, `protocol = "axi4"` only.

Monitor preset

TOML key ([bridge])	Type	Default	Effect
`mon_preset`	`"lite"` (also accepted: `"error_only"` /	`"lite"`	Accepted for compatibility; since 2026-09-26

TOML key ([bridge])	Type	Default	Effect
	`"functional"` / `"all"` / `"none"`)		every monitored port builds `axi_monitor_lite` through the `*_monlite` wrapper (error, timeout, completion and active-count threshold packets, same monbus, about a fifth of the full monitor's LUTs), so the cone presets no longer select hardware. Which packets a port emits is the runtime `cfg_*_enable` pins. To remove a monitor use `use_monitor = false` on the port or `use_no_monitors = true` on the bridge. `mon_add` / `mon_remove` are validated and ignored.

22.2.3 Derived Parameters

The generator works these out from the core set — you never write them yourself:

Derived Parameters

Parameter	Formula	Example
BRIDGE_ID_WIDTH	clog2(NUM_MASTERS), minimum 1	4 masters = 2 bits
MASTER_ID_WIDTH	widest master `id_width`	4
ID_PREFIX_WIDTH	clog2(NUM_MASTERS), 0 for one master	4 masters = 2 bits
XBAR_ID_WIDTH	MASTER_ID_WIDTH + ID_PREFIX_WIDTH	4 + 2 = 6 bits
STRB_WIDTH	DATA_WIDTH / 8	64b = 8 strobes

`BRIDGE_ID_WIDTH` sizes the sideband master index stored in each slave adapter's response tracker. `XBAR_ID_WIDTH` is the AXI ID width inside the fabric and on every AXI slave port, because each master adapter widens its IDs to `{BRIDGE_ID, id}`; a slave's `id_width` must be at least this. All four are emitted into `<bridge>_pkg.sv`. There is no `TOTAL_ID_WIDTH`.

22.2.4 Per-Port Configuration

Each master and slave port carries its own field set in the configuration file.

22.2.4.1 Master Port Configuration

Master Port Configuration

Field	Type	Description
name	string	Unique identifier
prefix	string	Signal prefix
channels	enum	"rw", "wr", or "rd"
data_width	int	Port data width
id_width	int	Port ID width
use_monitor	bool	Optional, defaults to `true`. Per-port monitor wrappers. Only meaningful on a `mon` variant; on a `no` variant there is nothing to disable.

22.2.4.2 Slave Port Configuration

Slave Port Configuration

Field	Type	Description
name	string	Unique identifier
prefix	string	Signal prefix
protocol	enum	`axi4`, `axi5`, `axil`, `axil5`, `apb`, `apb5`, `wb4` – the exact values `config_validator.valid_protocols` accepts. Note `axil`, not `axi4lite`.
axi5_features	list	On an `axi5` port: any of `nsaid`, `trace`, `mpam`, `mecid`, `unique` (droppable), `chunking` (droppable, 128-bit ports), `wakeup` (AWAKEUP, not path sideband), `poison`, `atomic`, `mte` (connectivity-gated; `mte` 128-bit ports). On an `axil5` port: `user` only. Classes and rules in HAS 4.4.
data_width	int	Port data width
base_addr	hex	Base address (must be 4K-aligned)
addr_range	hex	Address range size (must be multiple of 4K)
use_monitor	bool	Optional, defaults to `true`. Per-port monitor wrappers. Only meaningful on a `mon` variant; on a `no` variant there is nothing to disable.

22.2.4.3 Slave Address Window Rules (MANDATORY)

Every slave must occupy a 4K-aligned address window that is a multiple of 4 KB:

```
Rule 1: Base Address Alignment
  base_addr & 0xFFF == 0x000

  Valid:   0x00000000, 0x00001000, 0x80000000, 0xFFFF0000
  Invalid: 0x00000001, 0x80000800, 0xC0000100

Rule 2: Range Alignment
  addr_range % 0x1000 == 0
```

```
  Valid:   0x1000 (4 KB), 0x2000 (8 KB), 0x10000 (64 KB)
  Invalid: 0x800 (2 KB), 0x1001, 0x7FFF

Rule 3: Non-Overlapping Windows
  All slaves must have non-overlapping address ranges

  Valid:   Slave0: 0x00000000-0x0FFFFFFF, Slave1: 0x10000000-
0x1FFFFFFF
  Invalid: Slave0: 0x00000000-0x10000000, Slave1: 0x0FFFFFFF-
0x1FFFFFFF
```

Why This Rule Exists:

- Real memory systems use 4K page boundaries (virtual memory, MMU)
- Linker scripts and memory maps assume 4K granularity
- Address decoders simplify to bit masks with 4K alignment
- Prevents ambiguity in address decode logic

There's nothing arbitrary about the 4K requirement — it's the granularity the rest of the system already assumes.

22.2.5 Top-Level Monitor Configuration

When monitor collection is desired, the bridge TOML configuration includes:

Top-Level Monitor Configuration

Field	Type	Description
variants	list	List of bridge variants to generate. Supported: `["no"]` (no monitor), `["mon"]` (monitor only), `["no", "mon"]` (both). Default: `["no"]`

22.2.5.1 Monitor Identifiers

Each monitored port gets a unique `(UNIT_ID, AGENT_ID)` pair for identification in monitor packets:

```
UNIT_ID Assignment:
  - UNIT_ID = 1: Master-side monitor wrappers
(axi4_master_{rd,wr}_monlite)
  - UNIT_ID = 2: Slave-side monitor wrappers
(axi4_slave_{rd,wr}_monlite)

AGENT_ID Assignment (per port):
  - AGENT_ID = (port_index << 4) | channel_bit
  - channel_bit: 0 = read channel (AR/R), 1 = write channel (AW/W/B)
```

```
  - Example: Master port 2, write direction → AGENT_ID = (2 << 4) | 1
= 0x21
```

22.2.5.2 AXIL→Wider-Slave Master-Side Alignment

When an AXI4-Lite master connects to a wider AXI4 slave through the bridge (e.g., 32-bit AXIL master to 64-bit AXI4 slave), the generator emits a master-side alignment converter (the `axil_to_axi4_wide_align_{rd,wr}` modules) between the master adapter and the crossbar core. The converter handles partial-word alignment on the narrow side while preserving AXIL's single-beat semantics; the protocol stays AXIL — protocol shims are a slave-boundary concern, applied separately.

22.2.6 Parameter Validation

22.2.6.1 Generator Checks

The generator checks every configuration against these rules:

Generator Validation Checks

Check	Error Condition
NUM_MASTERS	< 1 or > 32
NUM_SLAVES	< 1 or > 256
DATA_WIDTH	Not power of 2
Address overlap	Slave ranges intersect
Connectivity	Master with no slaves
ID width	Insufficient for masters

22.2.6.2 Runtime Validation

Bridge includes optional assertions for:

- Address alignment
- Burst boundary crossing
- Protocol violations
- ID mismatch

22.3 Usage Example

A complete configuration is two parts: the TOML file that describes the bridge, and the CSV that describes the connectivity.

22.3.1 TOML Configuration

```
[bridge]
  name = "my_bridge"
  description = "Example bridge configuration"

  defaults.skid_depths = {ar = 2, r = 4, aw = 2, w = 4, b = 2}

  masters = [
    {name = "cpu", prefix = "cpu_m_axi", channels = "rw",
     id_width = 4, addr_width = 32, data_width = 64, user_width = 1},
    {name = "dma", prefix = "dma_m_axi", channels = "rw",
     id_width = 4, addr_width = 32, data_width = 256, user_width = 1}
  ]

  slaves = [
    {name = "ddr", prefix = "ddr_s_axi", protocol = "axi4",
     id_width = 6, addr_width = 32, data_width = 512, user_width = 1,
     base_addr = 0x80000000, addr_range = 0x40000000},
    {name = "uart", prefix = "uart_apb", protocol = "apb",
     addr_width = 12, data_width = 32,
     base_addr = 0x00000000, addr_range = 0x00001000}
  ]
```

22.3.2 CSV Connectivity

```
master\slave,ddr,uart
cpu,1,1
dma,1,0
```

Two configurations to use as starting points:

22.3.3 Simple 2x2

```
[bridge]
  name = "bridge_simple"
  masters = [
    {name = "m0", prefix = "m0_axi", data_width = 64},
    {name = "m1", prefix = "m1_axi", data_width = 64}
  ]
  slaves = [
    {name = "s0", prefix = "s0_axi", base_addr = 0x0, addr_range =
0x10000000},
    {name = "s1", prefix = "s1_axi", base_addr = 0x10000000,
```

```
addr_range = 0x10000000}
  ]
```

22.3.4 Mixed Protocol

```
[bridge]
  name = "bridge_mixed"
  masters = [
    {name = "cpu", prefix = "cpu_axi", data_width = 64, channels =
"rw"}
  ]
  slaves = [
    {name = "mem",  prefix = "mem_axi",  protocol = "axi4", data_width
= 512,
     base_addr = 0x00000000, addr_range = 0x80000000},
    {name = "gpio", prefix = "gpio_apb", protocol = "apb",  data_width
= 32,
     base_addr = 0x80000000, addr_range = 0x00010000},
    {name = "uart", prefix = "uart_apb", protocol = "apb",  data_width
= 32,
     base_addr = 0x80010000, addr_range = 0x00010000}
  ]
```


---

23 Verification Strategy

23.1 Overview

Bridge gets verified at three levels — unit, integration, and system — and all of it runs on CocoTB with protocol BFMs doing the driving. This chapter lays out the test infrastructure, the execution model, the coverage targets, the debug hooks, and the regression schedule, then closes with the gaps we know about.

23.2 Testing

23.2.1 Verification Levels

23.2.1.1 Unit Level

Test individual Bridge components:

Unit Level Test Focus

Component	Test Focus
Master Adapter	Skid buffer, sideband bridge_id (IDs pass through)
Slave Router	Address decode, arbitration
Width Converter	Upsize/downsize correctness
Protocol Converter	AXI4 to APB timing
Response Router	ID extraction, routing

23.2.1.2 Integration Level

Test complete Bridge functionality:

Integration Test Coverage

Test Category	Coverage
Address routing	All master-slave paths
Arbitration	Contention, fairness
ID tracking	Response routing
Error handling	OOR, timeouts
Performance	Throughput, latency

23.2.1.3 System Level

Test Bridge in target system:

System Level Tests

Test Category	Focus
End-to-end	CPU to memory
Multi-master	DMA + CPU
Mixed protocol	AXI4 + APB
Stress	Maximum traffic

Test Category	Focus

23.2.2 Test Infrastructure

23.2.2.1 CocoTB Framework with Protocol-BFM-Only Driving

Bridge verification runs on CocoTB with protocol BFMs only: **all test stimulus is driven through protocol BFMs (AXI4Master, AXI4Slave, etc.) with no direct DUT signal manipulation**. Each slave port is backed by an in-memory model (`MemoryModel`), and checking is assertion-based (readback matches write) rather than routing-based.

```
from CocoTBFramework.components.axi4 import AXI4Master, AXI4Slave
from TBClasses.shared.tbbase import TBBase

class BridgeTB(TBBase):
    def __init__(self, dut):
        super().__init__(dut)
        self.masters = [AXI4Master(dut, dut.aclk, f"m{i}_axi") for i
in range(NUM_MASTERS)]
        self.slaves = [MemoryModel(dut, dut.aclk, f"s{i}_axi") for i
in range(NUM_SLAVES)]

@cocotb.test()
async def cocotb_test_basic_write(dut):
    tb = BridgeTB(dut)
    await tb.setup_clocks_and_reset()

    # Write transaction via BFM only
    await tb.masters[0].write(addr=0x1000, data=0xDEADBEEF)

    # Verify: readback matches write (memory-model assertion)
    readback = await tb.masters[0].read(addr=0x1000)
    assert readback == 0xDEADBEEF
```

23.2.2.2 Test Categories

Test Categories

Category	Description	Example
Basic	Single transactions	Read/write
Burst	Multi-beat transactions	16-beat burst
Concurrent	Multiple masters active	M0 + M1
Boundary Probe	Address window edges	Top/mid/bottom of each slave page

Category	Description	Example
Stress	Maximum utilization	Back-to-back
Error	Error conditions	SLVERR injection, timeout
Monitor	Monitor packet collection	Packet capture, IRQ assertion

23.2.3 Test Execution Model

23.2.3.1 Serial Execution

Bridge tests run **serially only** (no parallel execution via pytest-xdist). Each test function is named per-module to avoid cross-test cocotb function name pollution:

```
# Naming convention: test_bridge_{N}x{M}_{variant}_{scenario}
def test_bridge_1x2_rd_basic(request):        # Test name indicates 1
master, 2 slaves, read-only
def test_bridge_1x2_rd_monitor_smoke(request): # Monitor test on 1x2
def test_bridge_4x4_rw_concurrent(request):    # Concurrent test on
4x4
```

Cocotb test functions inside each file follow the same naming pattern but with `cocotb_test_*` prefix to avoid pytest collection conflicts.

23.2.3.2 Boundary Probe Testing

Address-window boundary testing probes the **top, middle, and bottom** of each slave's address window to catch address-decode logic errors at window edges:

```
for slave_idx, (base, window_size) in enumerate(slave_windows):
    # Probe bottom (base address)
    await master.write(base + 0x000, data)

    # Probe middle (arbitrary address within window)
    await master.write(base + window_size // 2, data)

    # Probe top (base + window_size - 1)
    await master.write(base + window_size - 1, data)
```

23.2.4 Coverage Goals

23.2.4.1 Functional Coverage

Functional Coverage Goals

Category	Target
Address paths	100% master-slave

Category	Target
	combinations via boundary probes
Transaction types	Read, write, burst, error (SLVERR)
Monitor modes	Enabled and disabled
ID values	Full ID range (representative)

23.2.4.2 Code Coverage

Code Coverage Goals

Metric	Target
Line coverage	>95%
Branch coverage	>90%
FSM coverage	100% state transitions
Toggle coverage	>80% signals

23.2.5 Debug Features

23.2.5.1 Waveform Capture

Bridge tests support VCD/FST waveform dumps:

```
pytest test_bridge.py --vcd=waves.vcd
gtkwave waves.vcd
```

23.2.5.2 Debug Signals

Bridge includes optional debug signals:

Debug Signals

Signal	Description
dbg_aw_grant	Current AW grant
dbg_ar_grant	Current AR grant
dbg_outstanding	Outstanding transaction count
dbg_state	FSM states

23.2.6 Regression Testing

23.2.6.1 Continuous Integration

CI Test Schedule

Stage	Tests	Frequency
Pre-commit	Basic sanity	Each commit
Nightly	Full regression	Daily
Weekly	Performance	Weekly

23.2.6.2 Test Matrix

Configuration Test Matrix

Config	Data Width	Protocol	Tested
2x2	64	AXI4	Yes
4x4	64	AXI4	Yes
4x4	256	AXI4	Yes
2x2	64	AXI4+APB	Yes
RAPIDS	512	AXI4+APB	Yes

23.2.7 Known Limitations

23.2.7.1 Test Gaps

Known Test Gaps

Gap	Status	Plan
APB burst stress	Partial	Phase 3
Width downsize stress	Partial	Planned
32-master config	Not tested	Low priority

23.2.7.2 Simulator Support

Simulator Support Status

Simulator	Status
Verilator	Fully supported
Icarus Verilog	Supported
Commercial (VCS, Questa)	Not tested

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

24 Synthesis and Implementation

24.1 Overview

Everything before this page is about what a generated bridge does. This page is about what it costs and how fast it closes, measured rather than estimated: a worked Vivado flow that takes any generated bridge through synthesis, placement and routing as a stand-alone block, the numbers it produced for a reference set of configurations on the two parts this repository's boards carry, and how to read and repeat them. Chapter 5.3 carries the resource tables; this chapter is the flow behind them.

24.2 The Flow

24.2.1 Where it lives

`projects/components/fabric-gen-ip/bridge/fpga/` is an FPGA build directory in the repository's standard shape (handbook `fpga/cmn-infra/build-flows`): a Makefile of variables that includes `make/fpga_flow.mk`, a `tcl/` directory the flow discovers, and `reports/`. It builds no bitstream. Its one script, `synth_only.tcl`, is a non-project batch run: read the bridge's filelist closure, synthesize the bridge as the top with no pins (Vivado's out-of-context mode), constrain its clocks, place and route, and write reports.

Synthesis flow knobs

Knob	Default	Meaning
BRIDGE	bridge_2x2_rw	any generated fixture; the top module and the filelist follow from the name
PART	xc7a100tcsg324-1	the Artix-7 on the Nexys A7; xc7k325tffg900-2 is the Kintex-7 on the Genesys 2
CLK_NS	10.0	the period every clock is constrained to

```
cd projects/components/fabric-gen-ip/bridge/fpga
make lint  BRIDGE=bridge_2x2_rw_cdc                # verilator on the
same closure, seconds
make synth BRIDGE=bridge_2x2_rw                    # one run, a few
minutes
make synth BRIDGE=bridge_4x4_rw PART=xc7k325tffg900-2 CLK_NS=6.667
bin/synth_sweep.sh                                 # the reference
set, both parts
bin/summary_table.py                               #
reports/summary.csv as the tables below
```

Each run leaves `reports/<bridge>__<part>/` (utilization after synthesis and after routing, a per-instance utilization breakdown, the timing summary, the twenty worst paths overall and register-to-register, the clock-interaction and CDC reports, and a `summary.txt`) and appends one line to `reports/summary.csv`. Two bridges, or one bridge on two parts, never overwrite each other. The flow's build lock serializes Vivado runs in the directory, so the sweep is deliberately one run at a time.

24.2.2 How the bridge is constrained

The flow reads the constraints off the bridge's port list, so a new fixture needs nothing written by hand:

- **Clocks.** Every port whose name ends in `aclk` gets a clock of `CLK_NS`. A single-domain bridge gets one; a bridge with CDC slave ports (HAS 4.5a) gets one per domain, and the domains are declared asynchronous to each other, which is exactly how a system would constrain it. The crossing itself is then checked by the CDC report, not by setup analysis across it.
- **Resets.** Every `*aresetn` input is a false path. Reset is asynchronous on assertion in every generated build (MAS 6.1) and deassertion is synchronized outside the bridge.
- **Data I/O.** Inputs arrive, and outputs must be valid, 30% of the period into the cycle, against the clock of the port they belong to. This is the ordinary out-of-context convention for a block whose neighbours are registered; it is not a property of the bridge, which is why the register-to-register slack is reported on its own and is the figure the tables below quote.
- **Defines.** `XILINX` is set so `bridge_cam` takes its distributed-RAM attributes; Vivado sets `SYNTHESIS` itself, which removes the simulation-only checkers in the crossbar and subtractive adapter.

24.2.3 Reading the summary line

`summary.csv` columns

Column	What it is
`luts`, `ffs`, `bram_tiles`, `dsps`	post-route utilization of the whole bridge
`wns_ns`	worst setup slack over every path, I/O budget included
`wns_reg2reg_ns`	worst setup slack between registers inside the bridge
`fmax_reg2reg_mhz`	`1000 / (CLK_NS - wns_reg2reg_ns)`: the clock the internal paths would close at, estimated at this constraint
`worst_logic_levels`	LUT depth of the worst internal path
`unrouted_nets`	must be 0; anything else is a failed run, not a slow one

The Fmax column is an estimate at the period you constrained. Vivado stops optimizing a path once it has margin, so a bridge that shows +2 ns of slack at 10 ns is not guaranteed to close at 8 ns until you run it at 8 ns. The handbook's rule applies (`fpga/cmn-infra/timing-closure`): when a run does fail, look at the magnitude, the clock interaction and the logic levels, in that order, before touching the RTL.

24.3 Measured Results

24.3.1 What the first run found

The first bridge through this flow, `bridge_2x2_rw` on the Artix-7 at 10 ns, failed timing by 22 ns with every functional test green. The worst path ran from a master adapter's tracking pointer, through the crossbar's AR arbitration and mux, into the slave adapter's CAM: 40 LUT levels, a 32 ns data path, 83% of it routing. The CAM's Mode-2 allocate computed a newcomer's ordering count as the largest count among the entries with the same ID, written as a loop over the 16 entries, and that loop synthesized to sixteen chained comparators hanging off the arbitrated ARID (MAS 4.1). The counts of one ID are always `0..k-1`, so the value is the number of matching entries, a popcount; the entry to retire, the one match with count 0, is one-hot, so its index and stored master are OR reductions. With that change the same bridge meets 10 ns.

The CAM ordering-count scan, before and after (2026-09-11)

`bridge_2x2_rw`, Artix-7 100T -1, 10 ns	Before	After
LUTs	5,824	4,615
FFs	3,253	3,253
WNS register-to-register (ns)	-22.27	+0.12
Worst logic levels	40	9
Fmax estimate (MHz)	31	101

No simulation, formal proof or lint had a view of this: the block was correct and its depth was invisible until synthesis. It is why the flow exists and why it runs in minutes.

24.3.2 The reference set

Ten generated fixtures cover the shapes the generator produces: the baseline 2x2, its registered-crossbar, QoS, QoS-plus-registered and CDC variants, a 4x4 with mixed widths, a shim-heavy mix, a 32-bit AXI5 pair, the 128-bit native-AXI5 2x2 with Memory Tagging and chunking on every port, and a 5x3 with channel-specific masters. Each is constrained at 100 MHz on the Artix-7 and 150 MHz on the Kintex-7, the clocks the Nexys A7 and Genesys 2 harnesses in this repository run their fabrics at. Every row has zero unrouted nets, zero block RAM and zero DSPs.

Reference set on the Artix-7 100T -1 (Nexys A7), constrained at 10 ns

Bridge	LUTs	FFs	BRAM	WNS reg-to-reg (ns)	Fmax est. (MHz)	Worst logic levels
`bridge_2x2_a`	4,441	3,325	0	+0.40	104.2	8

Bridge	LUTs	FFs	BRAM	WNS reg-to-reg (ns)	Fmax est. (MHz)	Worst logic levels
xi5						
bridge_2x2_axi5_native	8,007	6,213	0	+0.04	100.4	12
bridge_2x2_rw	4,615	3,253	0	+0.12	101.2	9
bridge_2x2_rw_cdc	4,851	3,413	0	+0.27	102.7	9
bridge_2x2_rw_pipe	5,646	4,227	0	+1.27	114.6	6
bridge_2x2_rw_qos	5,136	3,317	0	-2.91	77.4	14
bridge_2x2_rw_qos_pipe	5,494	4,291	0	+0.17	101.7	9
bridge_4x4_rw	30,278	29,508	0	-1.57	86.4	14
bridge_5x3_channels	19,243	16,932	0	-2.66	79.0	16
bridge_mix_a	6,456	5,697	0	-0.29	97.1	16

Reference set on the Kintex-7 325T -2 (Genesys 2), constrained at 6.667 ns

Bridge	LUTs	FFs	BRAM	WNS reg-to-reg (ns)	Fmax est. (MHz)	Worst logic levels
bridge_2x2_axi5	4,430	3,325	0	+0.69	167.4	9
bridge_2x2_axi5_na	7,969	6,213	0	+0.26	156.1	12

Bridge	LUTs	FFs	BRAM	WNS reg-to-reg (ns)	Fmax est. (MHz)	Worst logic levels
tive						
bridge_2x2_rw	4,612	3,253	0	+0.97	175.6	9
bridge_2x2_rw_cdc	4,842	3,413	0	+0.55	163.4	9
bridge_2x2_rw_pipe	5,647	4,227	0	+1.73	202.6	6
bridge_2x2_rw_qos	4,963	3,317	0	-0.18	146.0	13
bridge_2x2_rw_qos_pipe	5,475	4,291	0	+0.70	167.6	9
bridge_4x4_rw	29,719	29,508	0	+0.28	156.5	13
bridge_5x3_channels	19,035	16,932	0	-0.13	147.1	17
bridge_mix_a	6,375	5,697	0	+0.53	162.9	8

24.3.3 Reading the rows

- **Every 2x2 without QoS meets both targets**, AXI5 and CDC variants included. The CDC port adds one clock domain and about 240 LUTs and 160 FFs for its five async FIFOs, and the crossing does not enter the fabric's critical path.
- **The registered crossbar is the timing option.** `xbar_pipeline` costs about 1,000 LUTs and 1,000 FFs on the 2x2 and cuts the worst path from 9 levels to 6, the only row with more than a nanosecond of margin on the Artix-7.
- **Every Artix-7 miss is the same path**: a master adapter's request register, through the crossbar's arbitration and address/ID mux, into the slave adapter's CAM allocate (the ID compare against every entry, then the popcount), with routing at 75-80% of the delay. QoS adds its priority

compare in series with the grant and misses 10 ns by 2.9 ns; the wide 4x4 and the 5x3 miss by 1.6 and 2.7 ns on the same shape; the shim mix by 0.3 ns. `bridge_2x2_rw_qos_pipe` is the demonstration: the same QoS arbiter behind the registered crossbar meets 10 ns with margin (-2.91 ns becomes +0.17 ns, 14 levels become 9). On the Artix-7 at 100 MHz, a configuration with more than two masters, wide ports or QoS should be generated with `xbar_pipeline = true`; on the Kintex-7 at 150 MHz only QoS-without-pipeline and the 5x3 fall short, by fractions of a nanosecond.

- **The native-AXI5 fabric costs its data width, not its features.** `bridge_2x2_axi5_native` is the 32-bit AXI5 pair at 128 bits with mte and chunking added on every port: 8,007 LUTs and 6,213 FFs against 4,441 and 3,325, and it meets both targets (+0.04 ns on the Artix-7, 12 levels). The tag and chunk fields are a few bits beside a 128-bit beat; the wider beat is the cost.
- **Width steps set the size.** The 4x4 with 32/64/128/256-bit ports is six times the 2x2 in both LUTs and FFs: every converter carries a full wide beat, in each direction, per path.
- **No block RAM, no DSPs, anywhere.** The CAM and the bridge-id FIFOs are LUT arrays; there is no arithmetic.

24.4 Design Notes

24.4.1 What moves the numbers

- **Data width** sets the register count almost linearly: every skid buffer and every crossbar stage holds a full beat.
- **Masters x slaves** sets the crossbar's mux width and the arbiter count; the master adapters and slave adapters scale with their own counts.
- **`xbar_pipeline`** adds one 2-deep skid per slave-side channel (MAS 2.3), paid in registers, and returns it as the shorter worst path.
- **Protocol shims and width converters** are the expensive ports: an APB or AXI-Lite slave carries a converter plus its own monitor sandwich, a width step carries a converter each way.
- **CDC slave ports** add one async FIFO per channel (HAS 4.5a); the cost is the FIFO storage, and the crossing does not enter the fabric's critical path.

24.4.2 Before a board build

Run this flow first. It takes minutes, it needs no constraints file, and it answers "does this configuration fit and close on this part" before a harness, a clock wizard and a pinout are wrapped around it.

25 AXI3 Integration

25.1 Boundary-Conversion Architecture

The bridge crossbar is AXI4 internally. An AXI3 port is therefore converted at the boundary, not natively carried through the fabric:

- **AXI3 slave port** (`protocol = "axi3"` on a slave): the crossbar's AXI4 master face drives `axi4_to_axi3`, whose AXI3 master face presents the external slave pins.
- **AXI3 master port** (`protocol = "axi3"` on a master): the external AXI3 master drives `axi3_to_axi4`, whose AXI4 slave face feeds the ordinary AXI4 master timing wrapper and the rest of the adapter.

Both converters require the AXI3 data width and the paired AXI4 width to match; any width adaptation is the bridge's internal `dwidth_converter` or `axil_to_axi4_wide_align` path *behind* the protocol converter, exactly as it is for native AXI4 ports.

25.2 Converter-Sourced Constraints

The bridge does not enforce AXI3 semantics itself — it instantiates the converters and relies on their contracts. The constraints below are taken from the converters MAS:

1. **No write-data interleaving across IDs.** `axi4_to_axi3_wr` stamps every AXI3 W beat with the current AWID and does not support interleaved W IDs; `axi3_to_axi4_wr` accepts only one outstanding write transaction at a time and asserts `ASSERT_A3_WID_STABLE` if `WID` changes mid-burst or differs from the active AWID. The upstream AXI3 master must keep W beats in transaction order.
2. **Egress bursts longer than 16 beats are chopped.** `axi4_to_axi3_rd` and `axi4_to_axi3_wr` split AXI4 bursts into sequential AXI3 bursts of up to 16 beats each; `INCR` and `FIXED` bursts are advanced/regenerated, while `WRAP` bursts longer than 16 beats are deferred (assertion-rejected).
3. **QoS and REGION are absent on AXI3 pins.** `axi4_to_axi3` drops them; `axi3_to_axi4` drives them to zero.
4. **Same-width converters only.** `AXI_DATA_WIDTH` must be identical on both sides; the bridge places its existing width-adaptation logic behind the converter when the external AXI3 width differs from a target slave width.
5. **Lock mapping per the ARM AXI3 encoding** (ARM IHI 0022: `00` normal, `01` exclusive, `10` locked), as fixed by bridge GitHub #98:
   - `axi4_to_axi3`: `1'b0 → 2'b00`, `1'b1 → 2'b01`.
   - `axi3_to_axi4`: `2'b01 → 1'b1`; `2'b00`, `2'b10` (locked — AXI4 dropped locked accesses), and the reserved `2'b11` → `1'b0`.

See converters MAS §3.15 *AXI4 to AXI3 Converter* (`15_axi4_to_axi3.md`) and §3.16 *AXI3 to AXI4 Converter* (`16_axi3_to_axi4.md`) for the full contracts.

25.3 Validation Evidence

The following generated configurations exercise AXI3 on both boundaries and pass the bridge GATE and FUNC regression suites:

- `bridge_1x2_axi3`
- `bridge_2x2_axi3_masters`
- `bridge_2x2_axi3_slaves`
- `bridge_mix_e`

25.3.1 Configuration constraints

The bridge validator enforces these AXI3-specific rules at configuration time:

- AXI3 port data widths must be one of {32, 64, 128, 256}.
- CDC on an AXI3 port is rejected in this version.
- AXI3 masters require `id_width > 0`.
- AXI3 slave `id_width` must be at least the fabric's master-index-prefixed width (the same rule that applies to AXI4 slaves).

25.4 References

- ARM, *AMBA 3 AXI Protocol Specification*, ARM IHI 0022 — the AXI3 protocol authority; 4-bit `AxLEN`, 2-bit `AxLOCK`, write-data `WID`, absence of `QoS`/`REGION`. (Reference archive)
- ARM, *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022E — the AXI4 protocol authority. (Reference archive)